СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

МИКРОПРОЦЕССОРНЫЕ УСТРОЙСТВА СИСТЕМ УПРАВЛЕНИЯ

3 курс

Авторы: Голик С.Е., Путов А.В., Игнатьев К.В., Копычев М.М.

Преподаватели: Голик Станислав Евсеевич
Копычев Михаил Михайлович
Лавриновский Виктор Сергеевич
Илатовская Екатерина Вадимовна

(Аудиторные работы)

Санкт-Петербург 2026

1. Описание лабораторного стенда и инструментальных средств разработки прикладных программ

1.1. Описание лабораторного стенда

Лабораторный стенд представляет собой аппаратно-программный комплекс с набором объектов управления, датчиков и средств индикации, управляемых микроконтроллером (МК) Atmega128A, работающем на частоте 11059200 Гц (11,0592 МГц). На рис. 1.1 приведена схема передней панели лабораторного стенда.

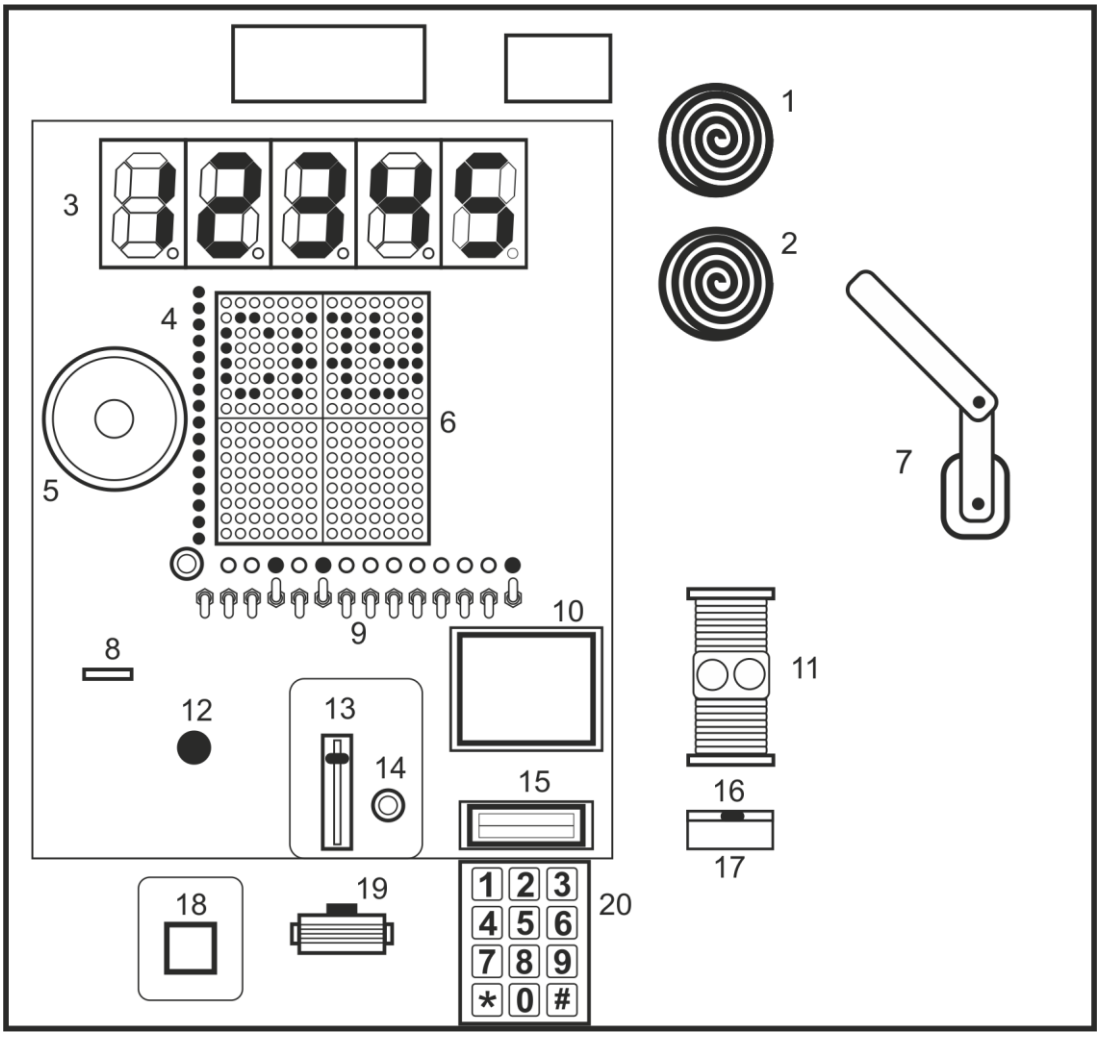


Рис. 1.1. Лабораторный стенд

В состав лабораторного стенда включены следующие электронные компоненты:

1. шаговый двигатель с силовым драйвером на базе микросхемы транзисторных ключей ULN2003A;
2. двигатель постоянного тока с силовым широтно-импульсным преобразователем на базе микросхемы L298N;

3. пятиразрядный цифровой дисплей на семисегментных индикаторах со схемой управления;
4. светодиодная шкала из 16 светодиодов, разделенная на две автономные части по 8 светодиодов в каждой;
5. динамический громкоговоритель, предназначенный для формирования звуковых сигналов;
6. светодиодная матрица 16x16 точек, состоящая из четырех квадратных матриц размера 8x8 светодиодов;
7. мини-манипулятор «серворука», состоящая из двух суставов с сервомашинкой в каждом суставе;
8. слот для карты памяти формата SD;
9. тумблеры для конфигурации стенда;
10. графический монохромный жидкокристаллический индикатор WG12864A размером 128x64 точки;
11. ультразвуковой дальномер HC-SR05;
12. многоцветный RGB светодиод для формирования световых сигналов разного цвета;
13. ползунковый потенциометр, используемый в качестве датчика;
14. инкрементный энкодер с кнопкой, используемый в качестве датчика угла поворота;
15. символьный монохромный жидкокристаллический индикатор WH1602C размером 2 строки по 16 символов в каждой;
16. магнитный подвес с катушкой, формирующей силовое магнитное поле;
17. датчик Холла магнитного подвеса;
18. тензометрический датчик для измерения момента;
19. цифровой температурный датчик DS1631S в связке с мощным резистором, используемым для изменения температуры;
20. мембранная клавиатура 4x3 клавиши.

Б*о*льшая часть перечисленных компонентов стенда используются в описываемых ниже лабораторных работах как подключенные к МК Atmega128A объекты управления, датчики и средства визуализации.

1.2. Введение в интегрированную среду разработки Atmel (Microchip) Studio

МК может быть запрограммирован с компьютера с установленной интегрированной средой разработки (IDE) и с помощью устройства, называемого программатором. Среда разработки предназначена для написания кода, компиляции и перевода языка СИ (например) в двоичный код, который может быть «понят» МК.

Среда разработки может использоваться одна из множества. Рассмотрим Atmel (Microchip) Studio (далее – STUDIO) (см. рис. 1.2).



Рис. 1.2. Atmel Studio (STUDIO)

STUDIO – основанная на Visual Studio бесплатная IDE для разработки приложений для 8- и 32-битных МК семейства AVR и 32-битных МК семейства ARM от компании Atmel (Microchip). STUDIO содержит компилятор GNU C/C++ и эмулятор, позволяющий отладить выполнение программы без загрузки в МК.

На первом этапе исходный код программы пишется на ассемблере или языке СИ. Этот этап выполняется с помощью текстового редактора, который является частью IDE, предназначенного для создания и изменения текста исходных программ и записи их в файл.

Перевод программы с СИ в машинные коды выполняется с помощью программного модуля транслятора. Транслятор не только переводит языковые команды в машинные инструкции, но и обнаруживает синтаксические ошибки в исходной программе с указанием их местоположения.

Следующий шаг – компиляция/линковка – несколько автономно транслируемых программных модулей объединяются (компилируются) в единый модуль для загрузки его в МК. Необходимость этой операции объясняется наличием ссылок в программных модулях. Эти ссылки относятся к другим программным модулям и модулям заголовочных файлов. Кроме того, в соответствии с информацией, помещенной транслятором в программный

модуль, компоновщик перемещает адреса этого модуля в соответствии с фактическим расположением программы в программной памяти МК. Компоновщик отслеживает ошибки сборки программного модуля и предоставляет информацию об этих ошибках.

STUDIO объединяет все описанные выше шаги разработки прикладной программы в единый рекурсивный процесс, в котором в любой момент можно быстро вернуться к любому предыдущему этапу, чтобы исправить ошибки или изменить алгоритм.

На рис. 1.3 показан пользовательский интерфейс STUDIO.

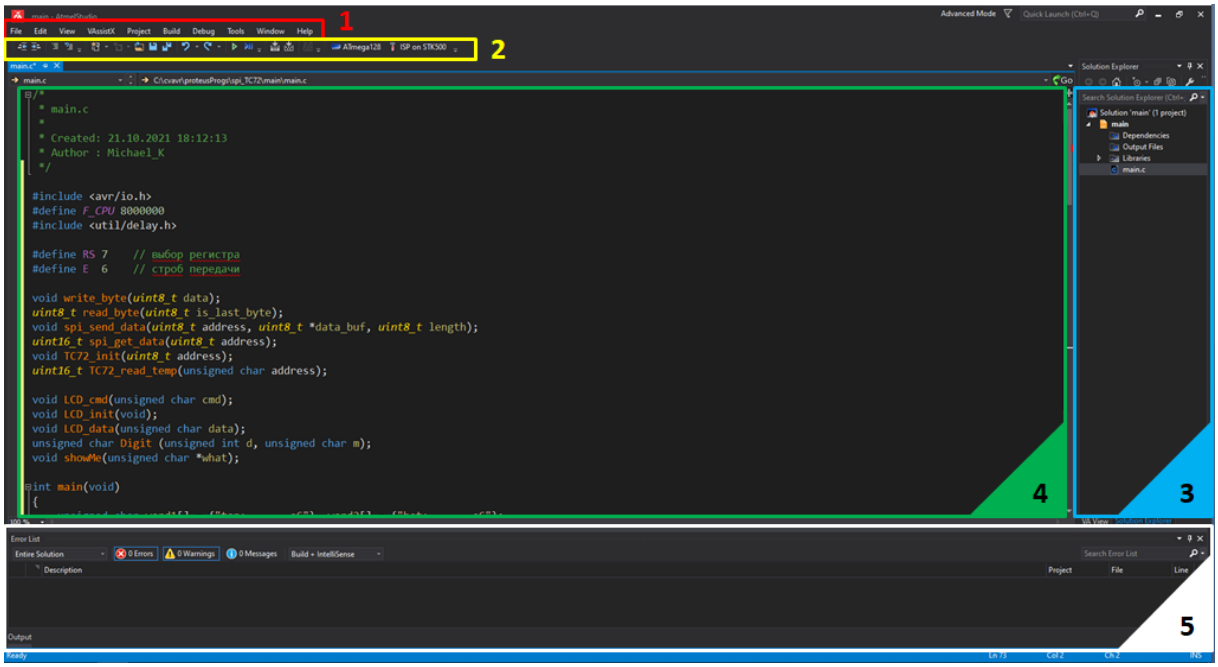


Рис. 1.3. Пользовательский интерфейс STUDIO

На рис. 1.3 цифра 1 – строка меню, 2 – кнопки управления (настраиваемые), 3 – дерево текущего проекта, 4 – текстовый редактор, 5 – информационное окно.

Для написания новой программы, необходимо создать новый проект. Для этого перейдите **File → New → Project** (см. рис. 1.4).

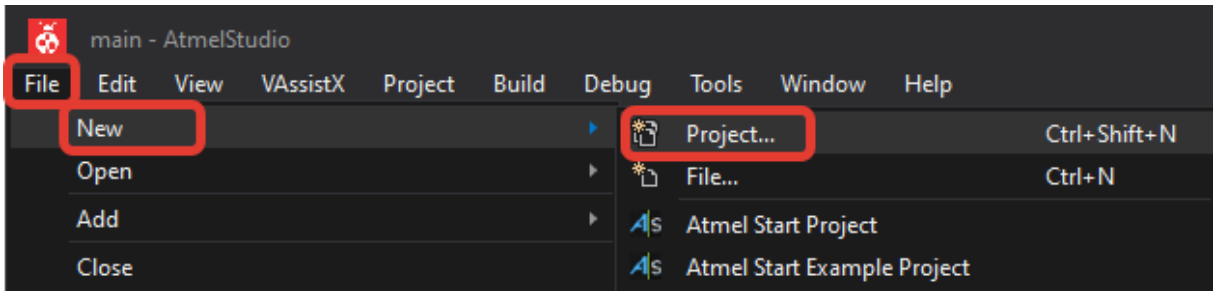


Рис. 1.4. Создание нового проекта

Откроется диалог, в котором будет несколько вариантов проектов на языке C/C++. Будем писать именно на языке СИ, выбираем **GCC C Executable Project** (см. рис. 1.5).

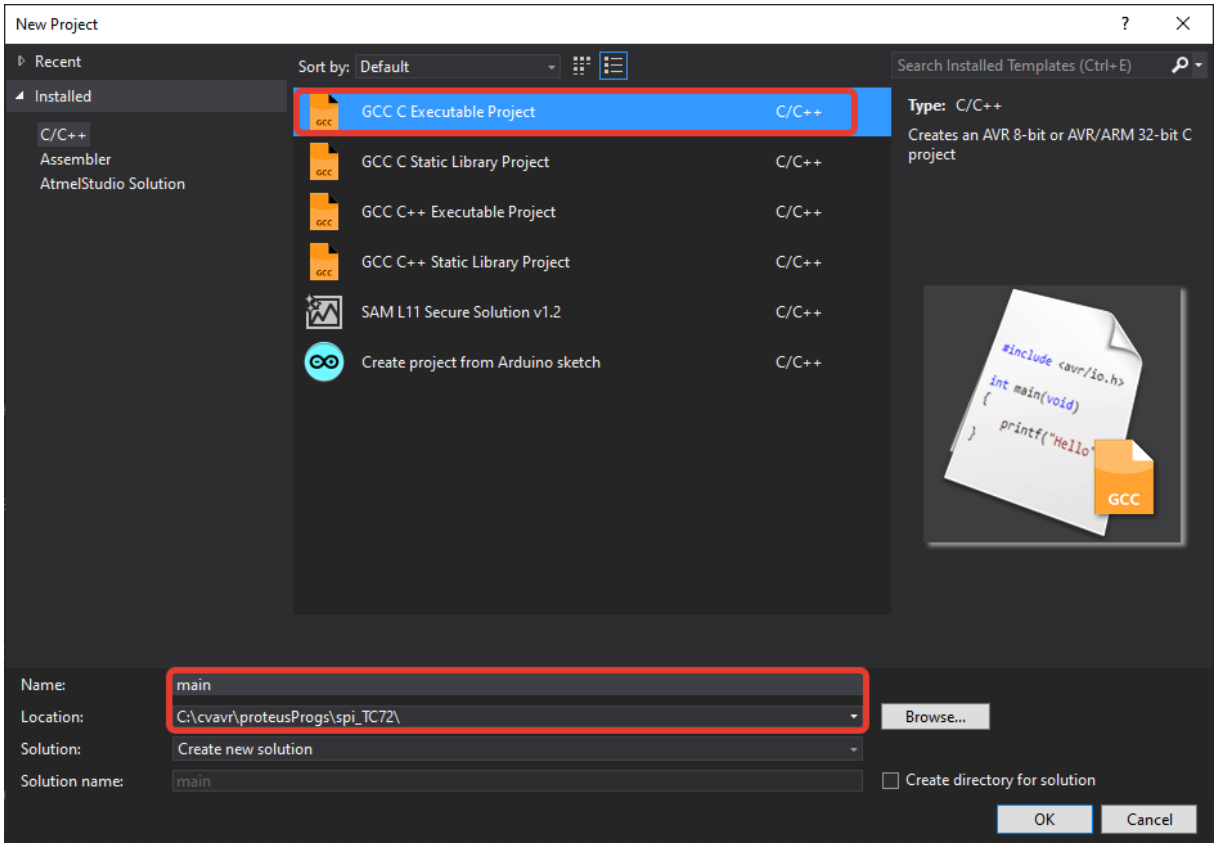


Рис. 1.5. Выбор типа проекта

В поле **Name** задаётся название файла проекта, в поле **Location** выбирается диск и папка, куда будет сохранён проект.

После этого создаётся новый файл проекта в указанном месте на жестком диске компьютера. См. рис. 1.6 и 1.7.

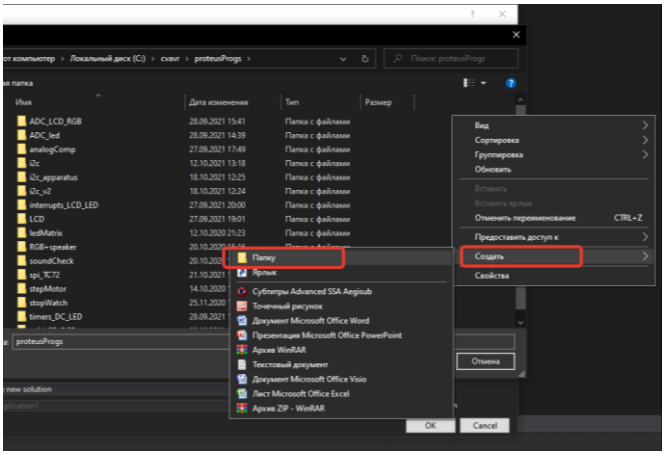


Рис. 1.6. Создавайте новую папку для каждого проекта

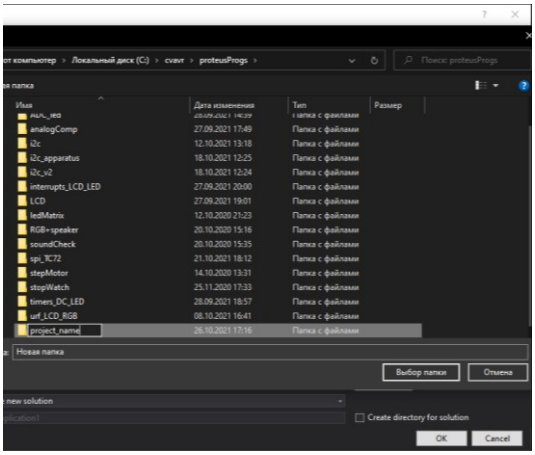


Рис. 1.7. Название проекта не должно содержать специальных символов

После создания папки, присвоения имени проекту и нажатия кнопки «ОК» появляется следующее окно (см. рис. 1.8). Теперь необходимо выбрать МК, для которого будет разрабатываться проект.

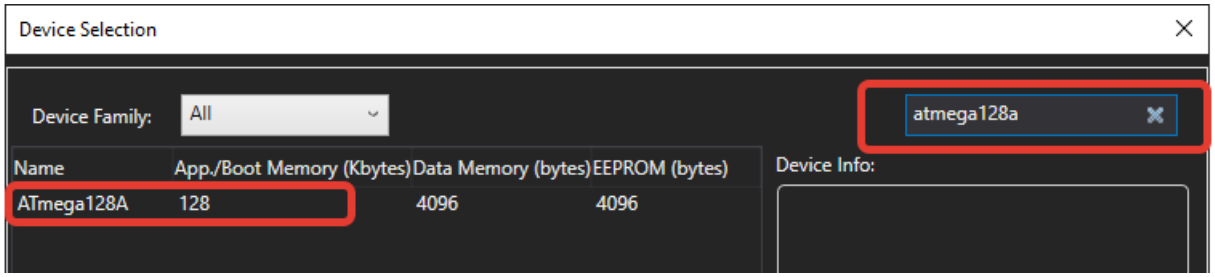


Рис. 1.8. Окно выбора МК

Мы работаем с Atmega128A, поэтому выбираем её. После этого появится похожее окно (см. рис. 1.9), в котором мы и будем писать нашу программу, отлаживать её и заливать на МК.

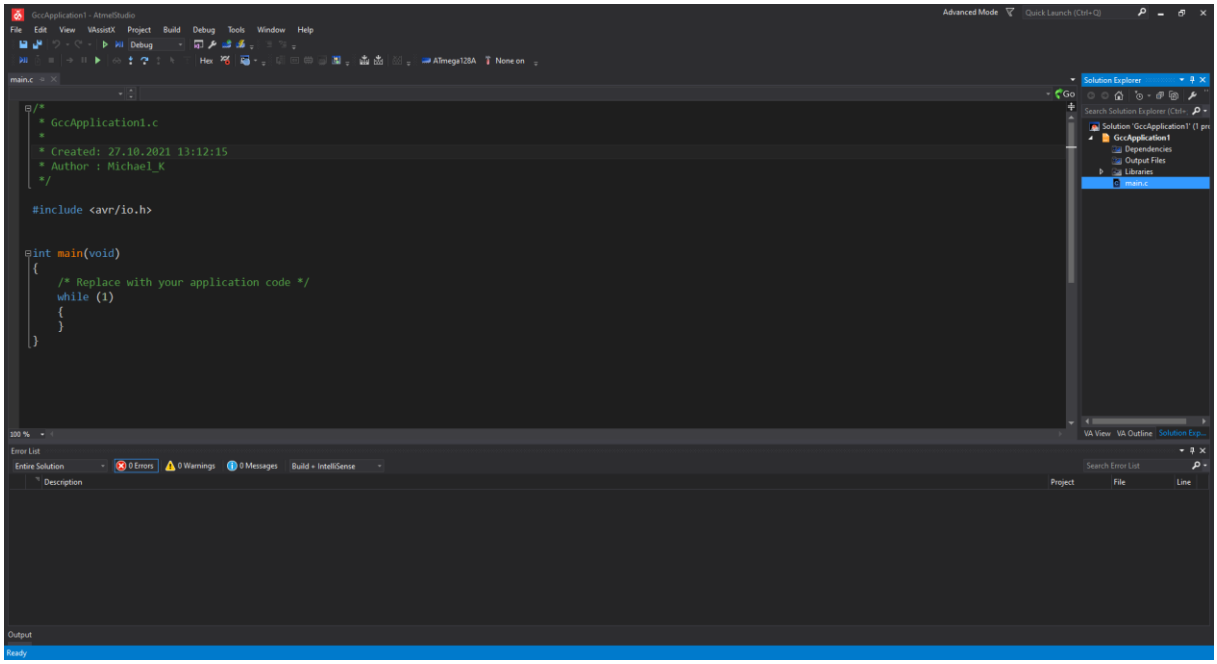


Рис. 1.9. Рабочий экран STUDIO

Немного настроим наш проект. Перейдите по вкладкам **Project** → ***project_name*** **Properties** (см. рис. 1.10).

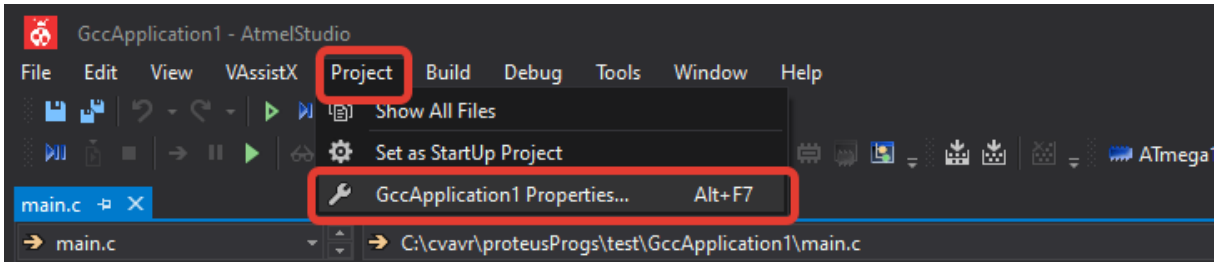


Рис. 1.10. Перейдем к свойствам проекта

В открывшемся диалоговом окне (см. рис. 1.11) перейдем на вкладку *Tool* и выберем свой программатор. После этого можно приступать к написанию программы.

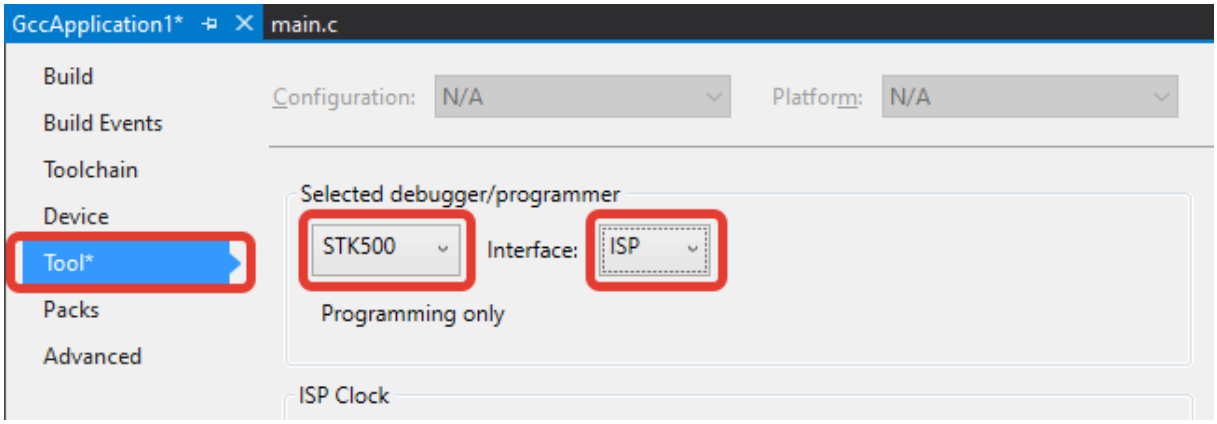


Рис. 1.11. Свойства проекта

После того, как код будет написан, его нужно собрать. Для сборки кода перейдите в **Project → Build Solution** (блок 1 либо пиктограмма в блоке 2 на рис. 1.3) или нажмите F7. Если в информационном поле проекта нет ошибок (в том числе предупреждений) (блок 5 на рис. 1.3) и пользователь считает, что код работает, можно загрузить программу в МК. Для этого следует перейти по вкладке **Debug → Start without Debugging** (блок 1 на рис. 1.3) или нажать комбинацию клавиш Ctrl + Alt + F5. Перед каждой прошивкой контроллера, если в проекте были сделаны какие-либо изменения, его обязательно нужно собирать.

1.3. Введение в редактор кода Visual Studio Code

Рассмотрим альтернативу STUDIO, а именно – редактор кода с элементами IDE Visual Studio Code (далее – VSCODE) (см. рис. 1.12).



Рис. 1.12. Visual Studio Code (VSCODE)

VSCODE – текстовый редактор, разработанный Microsoft для Windows, Linux и macOS. Позиционируется как «лёгкий» редактор кода для кроссплатформенной разработки веб- и облачных приложений. Включает в себя отладчик, инструменты для работы с Git, подсветку синтаксиса, IntelliSense и средства для рефакторинга (внесения изменений в код посредством специальных инструментов). Имеет широкие возможности для кастомизации: пользовательские темы, сочетания клавиш, файлы конфигурации и вспомогательные расширения.

Для того, чтобы комфортно писать программы под МК семейства AVR, следует установить несколько расширений для VSCODE:

- C/C++ – языковой пакет для возможности писать программы на языке СИ;
- AVR Helper – расширение, которое позволяет использовать компилятор AVR-GCC и программу для прошивки AVRDude;
- Serial Monitor – это расширение, позволяющее принимать и посылать данные через последовательный порт компьютера (нужно, например, для протокола UART).

Также на компьютере пользователя должны быть скачаны/установлены (а в случае использования ОС Windows – ещё и прописаны в PATH системы) следующие компоненты:

- Компилятор AVR-GCC – скачивается в виде папки с файлами, не устанавливается, помещается на жёсткий диск;
- Программа для прошивки МК AVRDude – скачивается в виде папки с файлами, не устанавливается, помещается на жёсткий диск;
- Приложение git (не обязательно) – нужно, чтобы использовать облачные репозитории, например – GitHub или GitLab.

На рис. 1.13 показан пользовательский интерфейс VSCODE. На рис.1.13 отмечены:

- «Список файлов» – дерево текущего проекта,
- «Поиск» – окно поиска и замены,
- «Git» – область работы с git-инструментами,
- «Отладчик» – утилита отладчика,
- «Расширения» – галерея расширений,

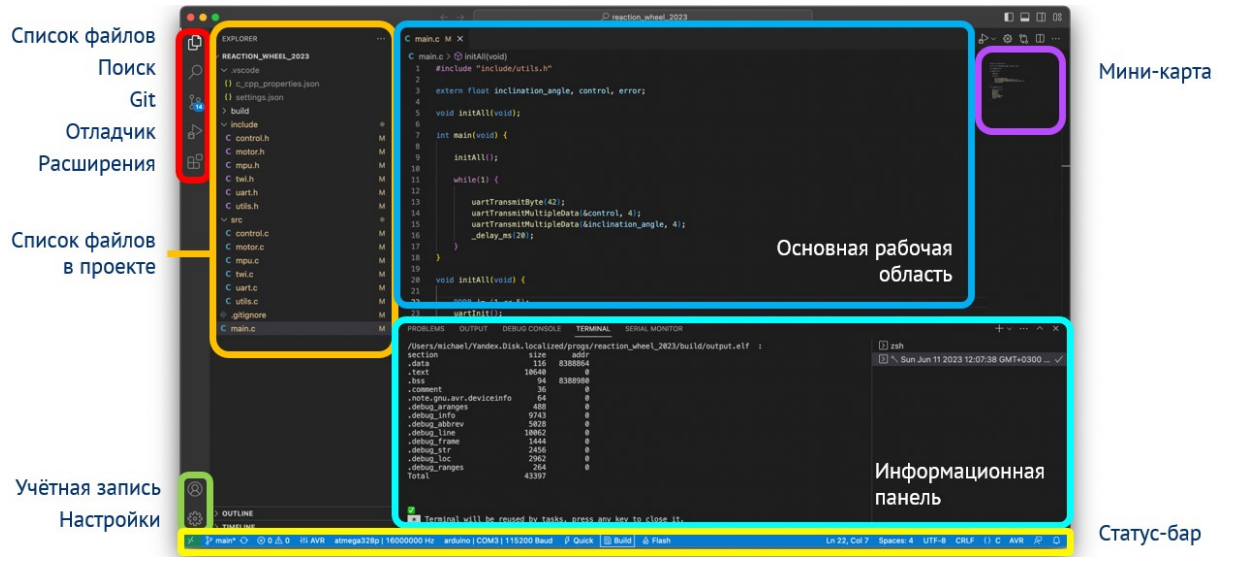


Рис. 1.13. Пользовательский интерфейс VSCODE

- «Учётная запись» – для возможностей синхронизации VSCODE на разных компьютерах,
- «Настройки» – меню настроек VSCODE и расширений,
- «Основная рабочая область» – редактор для написания кода,
- «Информационная панель» – область для вывода ошибок, предупреждений, также это область терминала,
- «Мини-карта» – код в миниатюре,
- «Статус-бар» – область, где появляются управляющие кнопки в зависимости от использования того или иного расширения.

Для написания любой новой программы, необходимо создать для этой программы новую папку. Для этого перейдите **File → Open Folder** (см. рис. 1.14) или нажмите комбинацию клавиш Ctrl + K Ctrl + O.

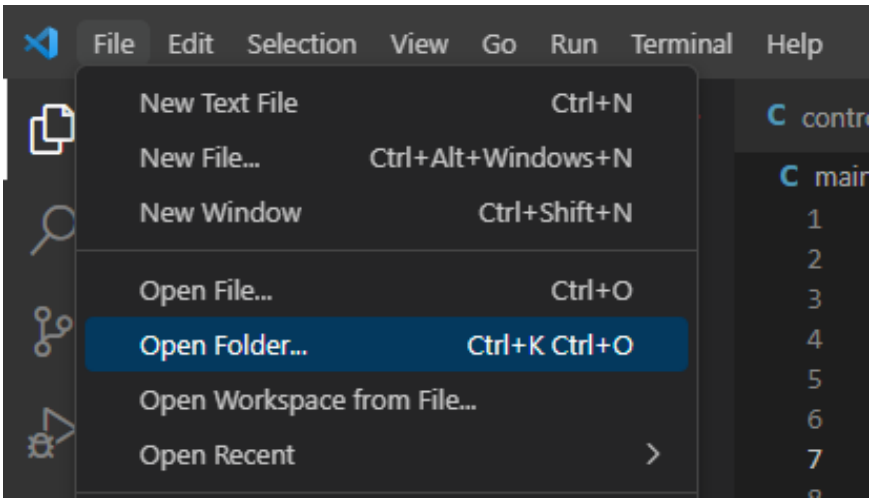


Рис. 1.14. Создавайте новую папку для каждой новой программы

Откроется стандартное диалоговое окно ОС, в котором следует перейти в каталог, где будет находиться папка с программой, далее – создать там новую папку (ПКМ → Создать → Папку), дать ей название и нажать кнопку выбора папки (см. рис. 1.15).

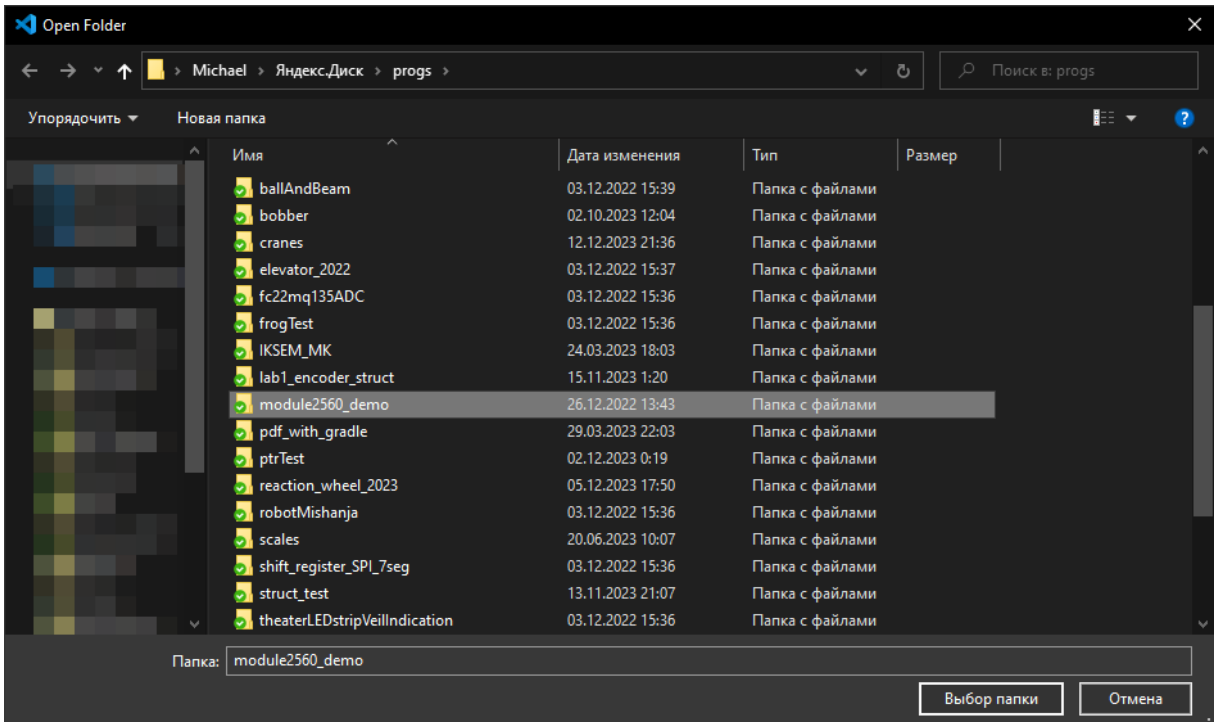


Рис. 1.15. Выбор папки проекта

В дереве проекта – пусто (см. рис 1.16). Надо заполнить его содержимым. Для этого нажмите на кнопку **New File…** (см. рис. 1.17). Поскольку в языке СИ существует одна точка входа в программу – файл **main.c** с функцией **`int main(void)`**, создадим файл **main.c** в котором будет наша главная функция (см. рис. 1.18). Обратите внимание, что не только содержимое файлов, но и их расширения прописываются руками при создании. Также можно создавать в дереве проекта папки (**New Folder…**) и наполнять их файлами.

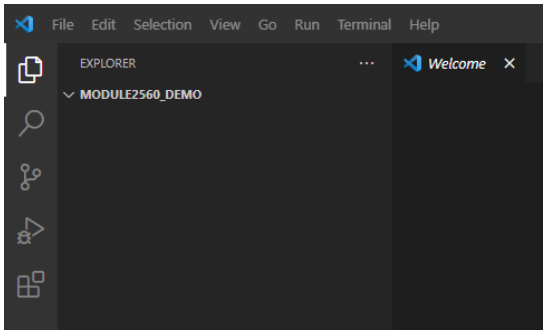


Рис. 1.16. В дереве проекта – пусто

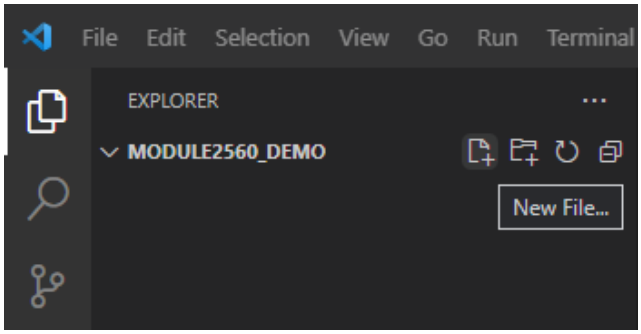


Рис. 1.17. Создаём новый файл в проекте

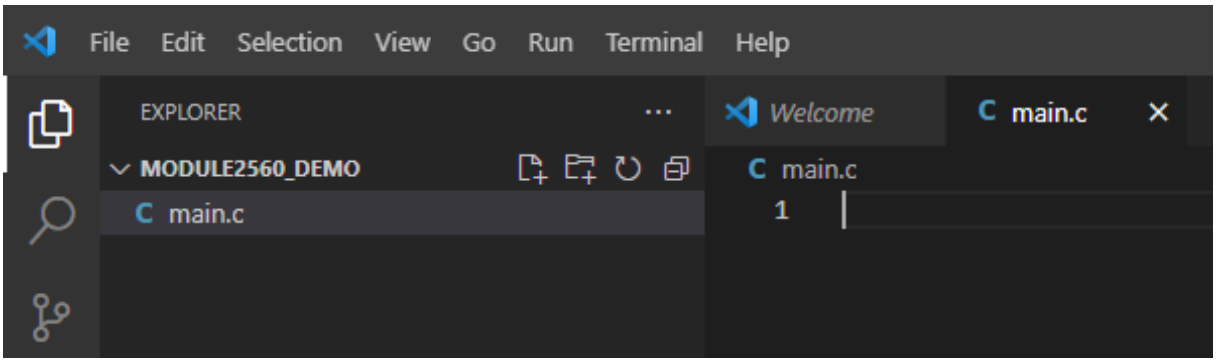


Рис. 1.18. Созданный файл **main.c**

Теперь нужно дать понять VSCODE, что программировать мы будем именно МК. Для этого обратимся к нижней панели VSCODE – к статус-бару. Жмём на кнопку вызова AVR Helper – **AVR** (см. рис. 1.19).



Рис. 1.19. Вызов расширения AVR Helper

После этого происходит настройка нашего проекта, а именно – создание папки **.vscode** в дереве проекта и последовательные запросы из т.н. **Command Palette** (см. рис. 1.20) о путях до AVR-GCC, шаг 1/4 (команда в командной строке **where avr-gcc**) и AVRDude, шаг 2/4 (команда в командной строке **where avrdude**). Шаги 3 и 4 можно пропустить нажатием Enter.

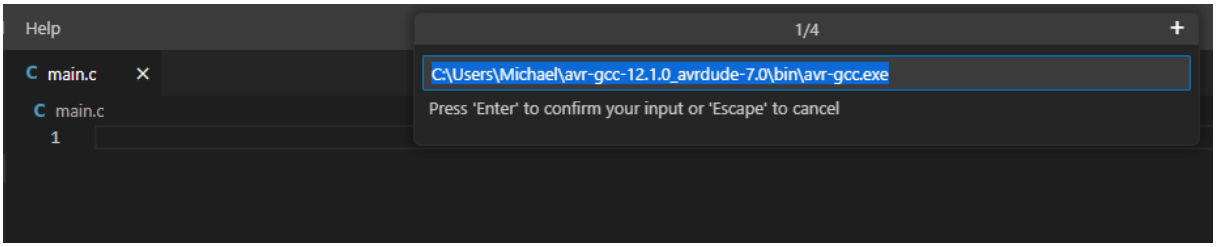


Рис. 1.20. Базовые настройки проекта (пути до компилятора и прошивальщика)

Далее нажимаем на следующую кнопку – см. рис. 1.21.

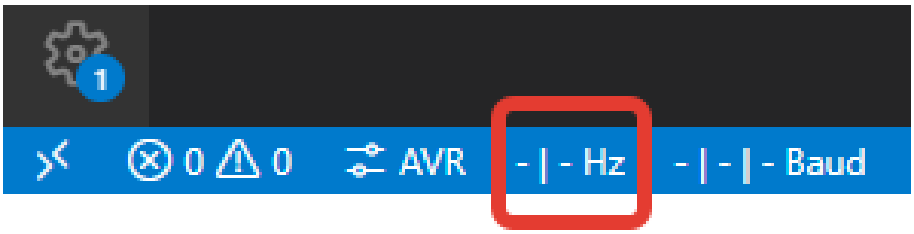


Рис. 1.21. Выбор МК и его частоты

Теперь в Command Palette нужно прописать целевой МК (например, **ATmega128A**) и его частоту работы (например, **11059200**). Обратите внимание, что теперь не нужно писать в коде **`#define F_CPU 11059200UL`**, теперь это значение будет забираться из настроек AVR Helper.

Далее нажимаем на следующую кнопку – см. рис. 1.22.

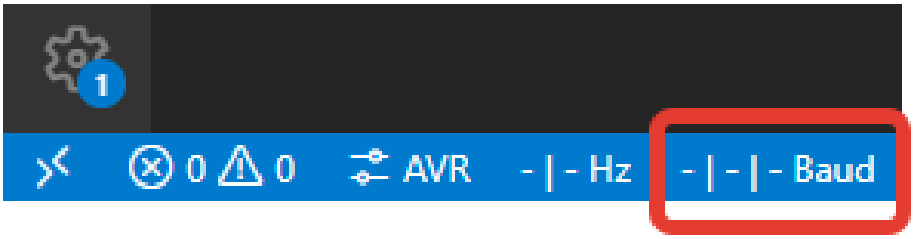


Рис. 1.22. Выбор программатора, его порта и его частоты передачи

Теперь в Command Palette нужно прописать тип программатора (например, **stk500**), порт, к которому он физически подключён (например, **COM5**, найти порт можно в диспетчере устройств) и частоту его работы (например, **115200**).

Все вышеописанные действия по настройке проекта могут быть автоматизированы и заполнены значениями по умолчанию, которые прописываются в настройках расширения AVR Helper.

После этих действий в статус-баре правее должны появиться три дополнительных кнопки (см. рис. 1.23), а именно – Quick – последовательная сборка проекта, его компиляция, проверка на ошибки и, если ошибки отсутствуют, прошивка МК, Build – только сборка проекта и проверка на ошибки и Flash – прошивка МК.



Рис. 1.23. Кнопки управления сборкой и прошивкой программы

После написания программы можно нажать на кнопку Build, в Command Palette появится информационное окно – ответьте нажатием Enter, после чего – если в проекте нет ошибок, нажать на кнопку Flash, в Command Palette опять появится информационное окно – ответьте нажатием Enter. Эти два действия объединены под одной кнопкой – Quick.

2. Быстрый старт

Методика Quick Start (Быстрый старт), введенная в обиход производителями современных средств разработки программ, ставит своей целью показать на простом примере основные шаги по использованию предлагаемой IDE (или – не IDE). Данная лабораторная работа имеет ту же цель, и поэтому воспользуемся этим приемом для получения первого представления и навыков работы на примере создания для МК Atmega128A программы вывода информации на светодиодную шкалу через параллельный порт.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 2.1.



Рис. 2.1. Состояние тумблеров конфигурации стенда

2.1. Описание лабораторной установки и примеры программ

Простейшим индикатором состояния различных объектов может служить светодиодная шкала, и поэтому первичное изучение работы параллельного порта в режиме вывода информации производится на примере программирования процедуры вывода сигналов на светодиодную шкалу для формирования различных световых эффектов. Фрагмент схемы лабораторного стенда, показывающий подключение светодиодной шкалы к параллельному порту PORTC, показан на рис. 2.2.

Светодиодная шкала представляет собой набор из восьми светодиодов, объединенных в виде линейной шкалы. Резисторы, включенные последовательно со светодиодами шкалы, предназначены для ограничения тока до номинального значения. Из представленной схемы нетрудно видеть, что для зажигания любого светодиода шкалы достаточно в соответствующий разряд порта PORTC записать значение логической единицы. При этом на выводе разряда порта появляется напряжение, зажигающее выбранный светодиод.

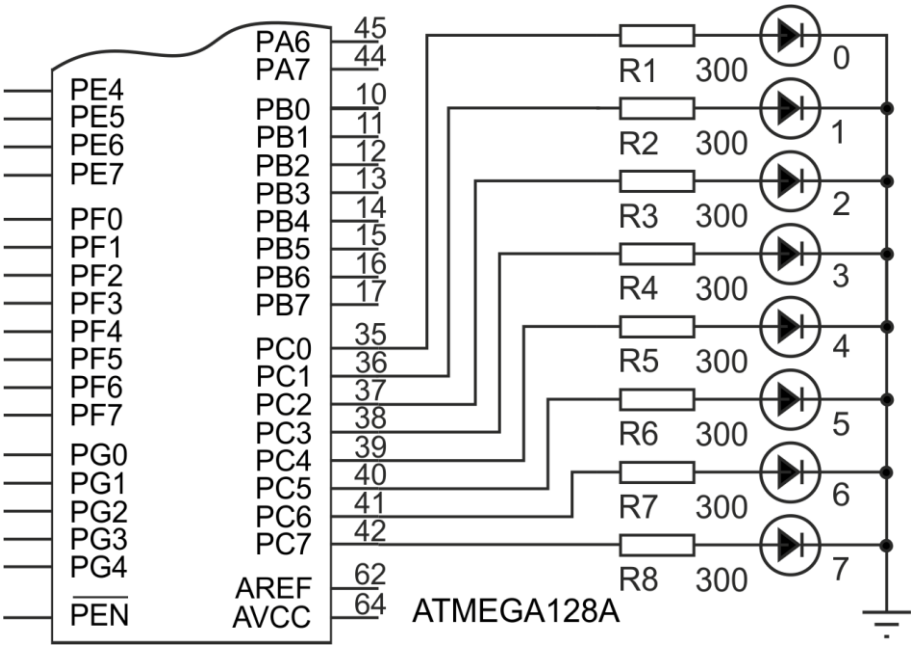


Рис. 2.2. Схема подключения светодиодной шкалы

Приведем пример программы, выполняющей подобное действие.

```
#include <avr/io.h>          // подключить заголовочный файл
#define F_CPU 11059200UL     // задать частоту тактирования

int main(void) {             // основная программа
    DDRC = 0xFF;             // все выводы порта PORTC - выходы
    PORTC = 0x00;            // погасить все светодиоды
    PORTC = PORTC | (1 << 0);   // зажечь светодиод 1
    while(1);                // бесконечный цикл
}
```

Назначение операторов программы описано в комментариях, однако некоторые из них требуют уточнений.

В программе используются символьные имена регистра данных порта PORTC и регистра направления DDRC и они должны быть сопоставлены соответствующим этим регистрам физическим адресам с помощью описаний

```
sfrb DDRC = 0x14;
sfrb PORTC = 0x15;
```

Хотя в нашей программе этих описаний нет, но, тем не менее, при компиляции никаких ошибок выявлено не будет. Связано это с тем, что все символические имена регистров ввода-вывода определены в специальных заголовочных файлах, поставляемых для всех поддерживаемых МК. Вся информация из этих файлов включается в программу с помощью директивы **`#include`**, что и сделано в первой строке нашей программы. Файл **`avr/io.h`**

и содержит сопоставление символических имен с адресами и другую информацию, относящуюся к МК семейства AVR.

Организация бесконечного цикла (в данном случае с помощью оператора **while**) характерна для большинства программ встроенных систем. Дело в том, что при отсутствии подобного цикла МК, выбрав из памяти последнюю команду программы и закончив ее выполнение, автоматически занесет в счетчик команд адрес следующей ячейки памяти программ, выберет хранящийся в ней код и начнет его выполнение. Естественно, что подобная попытка выполнения кода, не относящегося к рабочей программе, чревата непредсказуемыми последствиями.

В общем случае при написании программ следует использовать структурный подход, при котором часто повторяющиеся или самостоятельные фрагменты программы оформляются в виде функций. При таком подходе структура нашей программы может быть представлена в следующем виде:

```
#include <avr/io.h>          // подключить заголовочный файл
#define F_CPU 11059200UL     // задать частоту тактирования
#define D_LED DDRC           // рабочий порт направления
#define P_LED PORTC          // рабочий порт данных
//[]-----------------------------------------------------[]
//| Назначение: зажигание светодиода с номером n          |
//[]-----------------------------------------------------[]
void led(uint8_t n) {        // определение функции
    D_LED = 0xFF;            // все выводы P_LED - выходы
    P_LED = P_LED | (1 << n);   // зажечь светодиод n
}
int main(void){              // основная программа
    P_LED = 0x00;            // погасить все светодиоды
    led(0);                  // зажечь светодиод 0
    while(1);                // бесконечный цикл
}
```

Естественно, что данная программа настолько проста, что использование в ней функции никаких преимуществ не дает, но в более сложных программах такой подход упрощает реализацию алгоритма функционирования и значительно облегчает процедуру отладки.

При выполнении заданий необходимо учитывать, что скорость выполнения команд МК слишком высока, и для визуального восприятия включенного и выключенного состояний светодиода следует вводить паузу между моментами переключения. В наиболее простом случае функция временной задержки может представлять собой пустой цикл, выполняющийся

заданное число раз. Однако задать точное время задержки, реализуемой таким способом, довольно трудно. Более рациональным является использование таймера, имеющегося в практически любом МК, с помощью которого легко задать точное время задержки в широком временном диапазоне.

В стандартной библиотеке имеются две функции задания временных задержек (заметим только, что задержка может быть задана только числовым аргументом):

```
void _delay_us(double n);   // задержка в микросекундах
void _delay_ms(double n);   // задержка в миллисекундах
```

Прототипы данных функций заданы в заголовочном файле util/delay.h, и при их использовании в программе необходимо подключить этот файл с помощью директив

```
#define F_CPU 11059200UL    // задать частоту тактирования
#include <util/delay.h>     // подключить заголовочный файл
```

Составим теперь нашу первую тестовую программу, которая будет выполнять в бесконечном цикле следующие действия:

1. погасить все светодиоды;
2. сделать паузу 1 с;
3. зажечь нечетные светодиоды 1-3-5-7;
4. сделать паузу 1 с;
5. перейти к п. 1.

При составлении программы полагаем, что для зажигания комбинации светодиодов 1-3-5-7 в порт PORTC должна быть занесено двоичное число 0b10101010 (0xAA в шестнадцатеричном представлении).

Листинг программы, написанной с учетом заданного алгоритма, может быть представлен в следующем виде:

```
#include <avr/io.h>          // подключить заголовочный файл
#define F_CPU 11059200UL     // задать частоту тактирования
#include <util/delay.h>

//[]-------------------------------------------------[]
//| Назначение: зажигание светодиодов шкалы          |
//|  led - комбинация зажигаемых светодиодов         |
//[]-------------------------------------------------[]
void bar(uint8_t led) {      // определение функций
    DDRC = 0xFF;             // все выводы PORTC - выходы
    PORTC = led              // зажечь светодиоды
}
```

```
int main(void) {          // основная программа
  while(1) {              // бесконечный цикл
    bar(0x00);            // погасить все светодиоды
    _delay_ms(1000);      // пауза 1 секунда
    bar(0xAA);            // зажечь комбинацию
    _delay_ms(1000)       // пауза 1 секунда
  }
}
```

2.2. Варианты заданий

После успешного запуска первой программы, модифицируйте ее в соответствии с заданным преподавателем вариантом задания из перечисленного списка.

1. Реализуйте на светодиодной шкале эффект «бегущего огня». Для этого в бесконечном цикле последовательно зажигайте и гасите каждый светодиод шкалы с задержкой 1 с.

2. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с пары светодиодов 0-7, 1-6, 2-5, 3-4. После коммутации последней пары введите трехсекундную задержку.

3. В бесконечном цикле поочередно зажигайте и гасите с задержкой 1 с четверки светодиодов 0-3, 4-7.

4. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с светодиоды 0-2-4-6, 1-3-5-7.

5. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с тройки светодиодов 0-1-2, 1-2-3, 2-3-4, 3-4-5, 4-5-6, 5-6-7. После коммутации последней тройки введите трехсекундную задержку.

6. В бесконечном цикле зажигайте и гасите с задержкой 1 с пары светодиодов 3-4, 2-5, 1-6 и 0-7. После коммутации последней пары введите трехсекундную задержку.

7. В бесконечном цикле последовательно зажигайте все светодиоды шкалы с задержкой 1 с. После зажигания последнего светодиода погасите все светодиоды и повторите процедуру в обратном порядке.

8. В бесконечном цикле поочередно зажигайте и гасите с задержкой 1 с четверки светодиодов 0-2-4-6 и 1-3-5-7.

9. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с пары светодиодов 0-1, 1-2, 2-3, 3-4, 4-5, 5-6 и 6-7. После коммутации последней пары введите трехсекундную задержку.

10. В бесконечном цикле последовательно зажигайте каждый светодиод шкалы с задержкой 1 с. После зажигания последнего светодиода погасите все светодиоды и повторите процедуру.

11. В бесконечном цикле зажгите все светодиоды и затем последовательно гасите каждый светодиод шкалы с задержкой 1 с. После гашения последнего светодиода повторите процедуру.

12. В бесконечном цикле зажгите все светодиоды и затем последовательно гасите пары светодиодов 0-1, 2-3, 4-5, 6-7 шкалы с задержкой 1 с. После гашения последней пары повторите процедуру.

3. Лабораторные работы

3.1. Содержание отчета

Отчет по лабораторной работе должен содержать следующие разделы:

- задание на лабораторную работу;
- принципиальную электрическую схему, относящуюся к заданию;
- блок-схему алгоритма программы;
- текст программы на языке СИ с обязательными комментариями.

3.2. Программирование процедур ввода-вывода информации с использованием параллельных портов

Данные, представленные в виде двоичных логических сигналов, являются одним из типов данных, наиболее часто используемых МК для обмена информацией с внешними объектами. Поэтому каждый МК имеет линии ввода-вывода таких данных, подключенные к внешним пинам. С помощью этих линий МК взаимодействует с внешними датчиками и управляет исполнительными устройствами.

Помимо обмена логическими данными через одиночные линии в МК используется и широко распространенный параллельный обмен информацией. Для этого одиночные линии ввода-вывода объединяются в параллельные восьмиразрядные порты ввода-вывода, подключенные к системной магистрали МК.

МК серии AVR в зависимости от модели имеют в своем составе от одного до семи параллельных портов ввода-вывода. Каждый разряд порта подсоединен к одному из выводов (ножек) микросхемы.

Порты являются двунаправленными, т.е. любую линию каждого порта можно независимо настроить как на ввод, так и на вывод. Каждый порт имеет в своем составе три расположенных в пространстве ввода-вывода регистра: регистр хранения выводимых данных PORTx, регистр направления данных DDRx и регистр входных данных PINx.

Регистр входных данных PINx обеспечивает только возможность чтения, фактически он считывает информацию напрямую с вывода МК. Регистры данных PORTx и направления данных DDRx обеспечивают возможность чтения и записи.

В том случае, если линия настроена на ввод, то для повышения помехоустойчивости имеется возможность подключения к линии ввода внутренних резисторов, подтягивающих ее к напряжению питания.

Бит	7	6	5	4	3	2	1	0	
	PINx7	PINx6	PINx5	PINx4	PINx3	PINx2	PINx1	PINx0	**PINx**
R/W	R	R	R	R	R	R	R	R	
Н.з.	-	-	-	-	-	-	-	-	

Содержимое регистров направления DDRx (Data Direction Register) определяет направление передачи данных по каждой линии порта. Если бит регистра DDRx установлен в 0, то соответствующая ему линия порта настраивается на ввод, в противном случае – линия используется для вывода цифровой информации.

Бит	7	6	5	4	3	2	1	0	
	DDx7	DDx6	DDx5	DDx4	DDx3	DDx2	DDx1	DDx0	**DDRx**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Если линия порта настроена на вывод данных, то ее состояние будет определяться содержимым соответствующего разряда регистра PORTx.

Бит	7	6	5	4	3	2	1	0	
	PORTx7	PORTx6	PORTx5	PORTx4	PORTx3	PORTx2	PORTx1	PORTx0	**PORTx**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Если же линия порта настроена на ввод данных, то установка в 1 соответствующего бита в регистре PORTx приведет к подключению к этой линии подтягивающего резистора.

Состояние линий порта, в зависимости от комбинации содержимого разрядов регистров DDRx и PORTx, приведено в табл. 3.1.

Таблица 3.1

DDRx	**PORTx**	**Вход/Выход**	**Состояние линии**
0	0	Вход	Третье состояние (Hi-Z)
0	1	Вход	Режим входа с подтягивающим резистором
1	0	Выход	Вывод логического нуля
1	1	Выход	Вывод логической единицы

Упрощенная схема одного разряда любого порта PORTxn представлена на рис. 3.1.

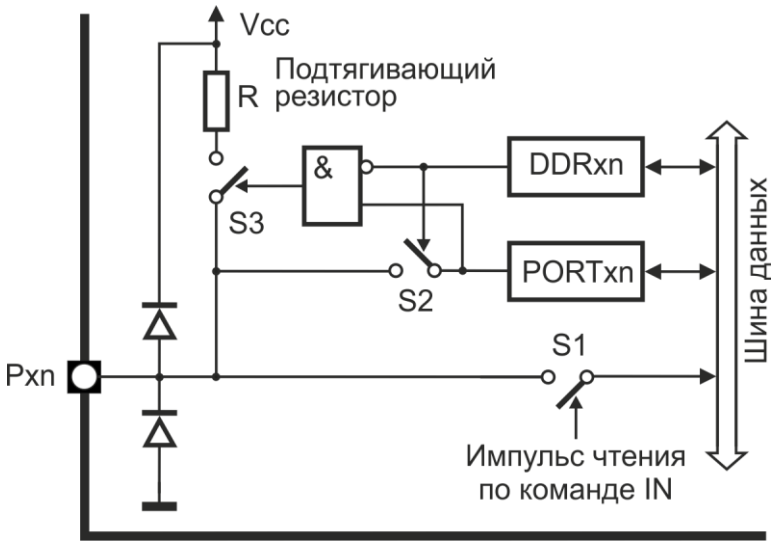


Рис. 3.1. Упрощенная схема одного разряда порта

Если разряд регистра направления DDRxn установлен в 1, то ключ S2 замкнут, разряд PORTxn порта настроен на вывод данных и состояние порта передается на ножку МК Pxn.

Для настройки заданного разряда порта PORTxn на ввод данных соответствующий разряд регистра направления DDRxn должен быть сброшен в 0. Это приводит к размыканию ключа S2 и разряд порта PORTxn отключается от вывода Pxn и его состояние не искажает считываемых данных.

Для считывания входных данных используется команда IN, при выполнении которой формируется импульс чтения, замыкающий ключ S1. При этом вводимые данные не сохраняются в разрядах порта, а считываются непосредственно с контактов Pxn в шину данных. Если разряд DDRxn = 0 и разряд PORTxn = 1, то на выходе элемента «И-НЕ» устанавливается уровень логической 1, замыкающий ключ S3, с помощью которого подтягивающий резистор R подключается к внешнему выводу Pxn. Для всех остальных комбинаций на выходе элемента «И-НЕ» устанавливается уровень логического 0 и резистор будет отключен.

Если разряд DDRxn = 0 и разряд PORTxn = 0, то вывод порта Pxn находится в отключенном состоянии (все три ключа разомкнуты).

После сброса МК или подачи напряжения питания, начальное значение всех регистров направления DDRx равно 0 и по умолчанию все порты настроены на ввод, а начальное значение любого регистра PORTx также равно 0 и по умолчанию все разряды портов будут находиться в состоянии высокого импеданса.

В заключение раздела рассмотрим несколько примеров.

Пример 1.

```
DDRA = 0xFF;  // настройка всех выводов порта A как выходов
PORTA = 0x0F; // вывод в разряды порта A значений 0b00001111
```

Пример 2.

```
uint8_t a;
DDRD = 0;     // настройка всех выводов порта D как входов
a = PIND;     // прочитать 8 выводов порта D в переменную a
```

Пример 3.

```
// настроить разряды 1, 3, 5 порта C на выход
DDRC = DDRC | (1 << 1) | (1 << 3) | (1 << 5);
// вывести в разряды 1 и 3 порта C логические 1
PORTC = PORTC | (1 << 1) | (1 << 3);
// вывести в разряд 5 порта C логический 0
PORTC = PORTC & ~(1 << 5);
```

ЛР № 1 Управление многоразрядным семисегментным индикатором

Цель работы. Вывод информации на семисементный индикатор с использованием параллельного порта (STUDIO).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.2.



Рис. 3.2. Состояние тумблеров конфигурации стенда для работы 1

Методические указания по выполнению работы

Для отображения цифровой и простейшей символьной информации в микропроцессорных системах очень часто используются светодиодные сборки, называемые семисегментными индикаторами. Как следует из его названия, индикатор состоит из семи сегментов, ориентированных таким образом, что подсветка различных комбинаций сегментов позволяет формировать алфавитно-цифровые символы. Каждый сегмент подсвечивается своей цепочкой светодиодов с возможностью индивидуального включения сегмента. Поскольку в первую очередь индикатор предназначен для отображения цифр, то в его состав включена также десятичная точка. Внешний вид и схема включения светодиодов всех сегментов индикатора показаны на рис. 3.3.

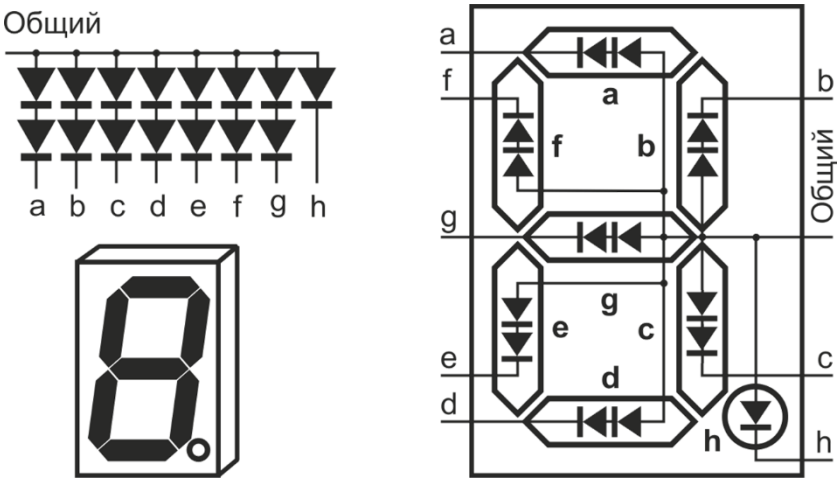


Рис. 3.3. Семисегментный индикатор

В лабораторном стенде используется набор из пяти семисегментных индикаторов (поз. 3 на рис. 1.1), позволяющий выводить пятиразрядные числа.

Каждый разряд такого числа последовательно, один за другим, выводится с помощью одного порта МК на соответствующий индикатор.

Однако для вывода числа на индикатор необходимо подавать на него не само число, а код его отображения на индикаторе. Например, для вывода числа 3 необходимо подать высокий уровень сигнала на входы a, b, g, e и d.

В случае, когда сегменты индикатора a ÷ g подключены к разрядам используемого порта МК 0 ÷ 7 соответственно, то все коды отображения чисел на индикаторе можно представить в виде, сведенном в табл. 3.2. Последние строки в таблице необходимо заполнить самостоятельно.

Таблица 3.2.

№ ножки	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**Код**
Сегмент	**h**	**g**	**f**	**e**	**d**	**c**	**b**	**a**	**отображения**
Цифра «0»	0	0	1	1	1	1	1	1	0x3F
Цифра «1»	0	0	0	0	0	1	1	0	0x06
Цифра «2»	0	1	0	1	1	0	1	1	0x5B
Цифра «3»	0	1	0	0	1	1	1	1	0x4F
Цифра «4»	0	1	1	0	0	1	1	0	0x66
Цифра «5»	0	1	1	0	1	1	0	1	0x6D
Цифра «6»	0	1	1	1	1	1	0	1	0x7D
Цифра «7»	?	?	?	?	?	?	?	?	?
Цифра «8»	?	?	?	?	?	?	?	?	?
Цифра «9»	?	?	?	?	?	?	?	?	?

Для организации процесса последовательного вывода разрядов отображаемого числа и управления им используются специальные микросхемы − восьмиразрядные регистры-защелки 74HC573. Функция такой микросхемы состоит в том, что при подаче на вход LE (Latch Enable) управляющего сигнала высокого уровня в ее внутренний регистр записывается комбинация восьми входных сигналов D0…D7, затем, после подачи на LE сигнала низкого уровня, входные линии отключаются и введенная комбинация сохраняется (защелкивается) во внутреннем регистре. При подаче на второй управляющий вход OE (Output Enable) сигнала низкого уровня, эта предварительно запомненная комбинация входных сигналов выводится на выходные линии микросхемы-защелки Q0…Q7. В схеме лабораторного стенда ножка OE заземлена, то есть всегда имеет низкий уровень, и, следовательно, управление производится только уровнем сигнала на выводе LE.

Схема подключения индикаторов к МК показана на рис. 3.4.

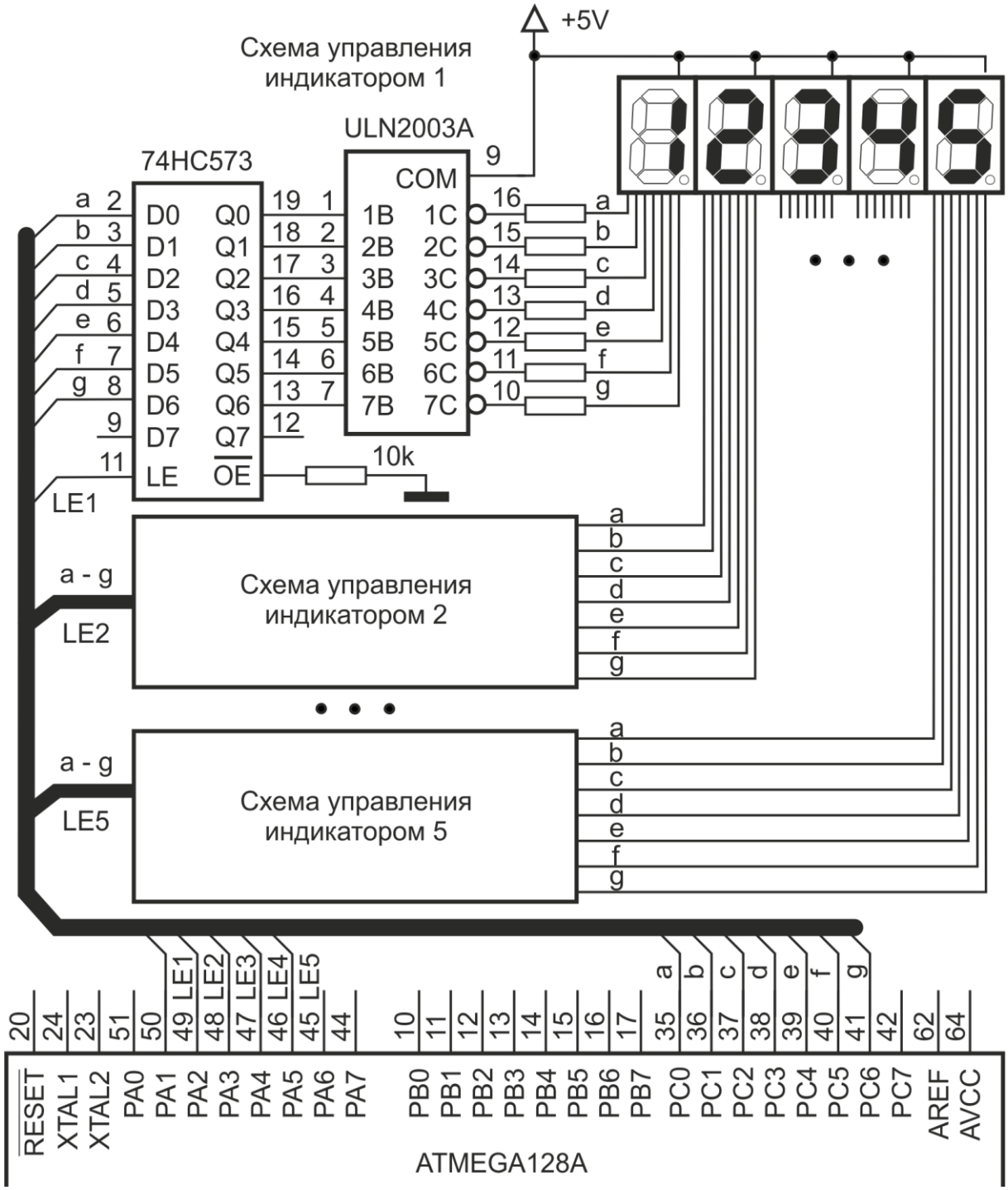


Рис. 3.4. Схема включения семисегментных индикаторов

В стенде используются семисегментные индикаторы, в каждом сегменте которых расположена цепочка из нескольких последовательно соединенных светодиодов, вследствие чего для зажигания сегмента требуется ток в несколько десятков миллиампер, недопустимый для выходных линий регистра-защелки. Поэтому в схеме предусмотрено включение между регистром-защелкой и

индикатором силового драйвера – микросхемы ULN2003, предназначенной для подключения устройств с сильноточной нагрузкой. В данном случае драйвер является инвертирующим – при подаче на его вход логической единицы на выходе формируется логический ноль. Графически инверсия иллюстрируется наличием небольшого кружка на выходной линии драйвера.

Как видно из схемы, входные линии всех защелок подключены к линиям параллельного порта PORTC, через который выводится код отображаемого символа. Входы LE регистров-защелок 1…5 подсоединены к линиям PA1…PA5 порта PORTA соответственно. Такое включение индикаторов позволяет последовательно выводить через порт PORTC код символа отображаемого разряда десятичного числа, затем стробом 0-1-0, подаваемым в одну из линий порта PA1…PA5 запоминать этот код в соответствующем отображаемому разряду регистре-защелке. После окончания процедуры заполнения регистров-защелок для всех пяти индикаторов на каждом из них будет отображен символ, код которого был ранее сохранен в его защелке.

Из логики работы используемой схемы включения следует, что вывод на индикатор с пятью десятичными разрядами любого целого числа производится поразрядно и для выполнения этой операции необходимо предварительно выделить значение цифры в каждом разряде выводимого числа.

Вычислить значения цифр в разрядах пятиразрядного десятичного числа поможет следующая функция, которую можно без изменений использовать в разрабатываемых программах.

```
//[]-------------------------------------------------[]
//| Назначение: выделение цифр из разрядов пятиразрядного|
//|  десятичного положительного числа                   |
//| Входные параметры:                                  |
//|  d - целое десятичное положительное число           |
//|  m - номер разряда (от 1 до 5, слева направо)       |
//| Функция возвращает значение цифры в разряде m числа d|
//[]-------------------------------------------------[]
uint8_t digit(uint16_t d, uint8_t m) {
  uint8_t i = 5, a;
  while(i){                  // цикл по разрядам числа
    a = d%10;                // выделяем очередной разряд
    if(i-- == m) break;      // выделен ли желаемый разряд
    d /= 10;                 // уменьшаем число в 10 раз
  }
  return(a);
}
```

Примеры использования функции:

```
uint16_t f = 7236;
uint8_t k = digit(f,5);     // функция возвращает k = 6
uint8_t k = digit(f,2);     // функция возвращает k = 7
uint8_t k = digit(f,1);     // функция возвращает k = 0
```

Поскольку двоичный код отображаемой цифры не совпадает с ее байтом индикации, то в программе должна осуществляться перекодировка в соответствии с таб. 3.2, ставящей в соответствие двоичный код индицируемой цифры своему байту индикации, который и выдается в выходной порт для отображения цифры на индикаторе.

С учетом сказанного, для формирования цифр, отображаемых на пятиразрядном семисегментном индикаторе стенда, нужно выполнить для k = 1 ÷ 5 следующую последовательность действий:

1. с помощью функции digit получить цифру k-го десятичного разряда выводимого числа;
2. по информации, представленной в таб. 2.2, сформировать код отображения полученной цифры;
3. вывести код отображения в порт PORTC;
4. защелкнуть код отображения в регистре-защелке k, для чего подать сигнал LE = 1, записав 1 в разряд k порта PORTA, сделать паузу 1 мкс и подать 0 для выдачи сигнала LE = 0;
5. если k = 5, то закончить, иначе – перейти к п. 1.

Варианты заданий

Рассчитать и с паузой в 1 сек вывести последовательно на индикатор 20 членов числовой последовательности *x(i)*, сформированной по формуле арифметической прогрессии:

$$x(i+1) = x(i) + d$$

для вариантов заданий, представленных в табл. 3.3.

Таблица 3.3

№ варианта	*x(0)*	*d*	№ варианта	*x(0)*	*d*
1	1	4	7	3	3
2	2	3	8	4	4
3	3	2	9	4	2
4	4	1	10	3	1
5	1	1	11	2	3
6	2	2	12	1	4

Алгоритм работы приведён на рис. 3.5. Незначащие нули слева – убрать.

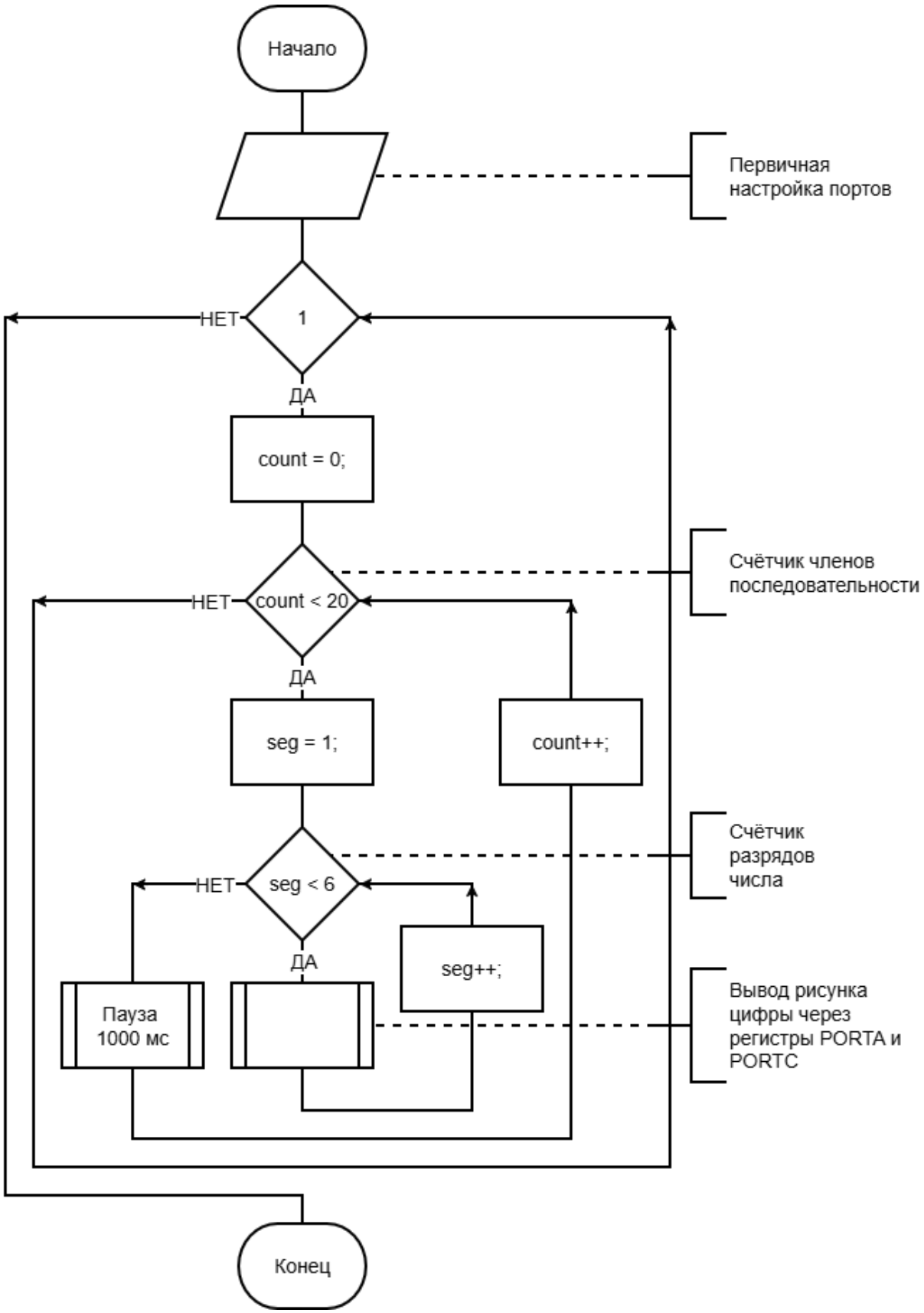


Рис. 3.5. Алгоритм вывода последовательности чисел на индикаторы

Проект рекомендуется реализовывать в STUDIO.

ЛР № 2 Динамическое управление светодиодной матрицей с использованием таймеров

Цель работы. Вывод информации на светодиодную матрицу с использованием аппаратных таймеров/счётчиков МК (STUDIO).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.6.



Рис. 3.6. Состояние тумблеров конфигурации стенда для работы 2

Методические указания по выполнению части работы с матрицей

Светодиодная матрица (Dot Matrix) представляет собой прямоугольный корпус с набором точек, каждая из которых состоит из одного светодиода. С помощью такой матрицы можно легко формировать алфавитно-цифровые символы, а также всевозможные пиктограммы. Внешний вид и схема части, используемой в работе (поз. 6 на рис. 1.1) матрицы приведены на рис. 3.7.

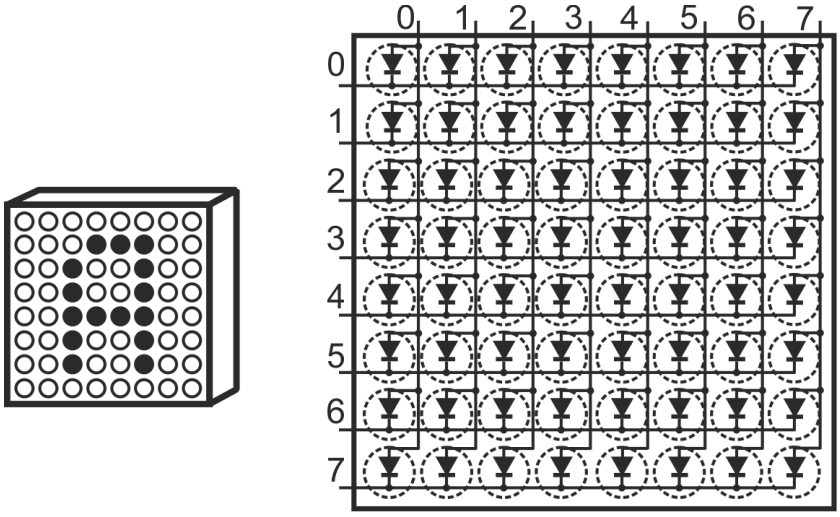


Рис. 3.7. Светодиодная матрица 8x8

Рассмотрим подключение одной светодиодной матрицы из четырех, для которой номер активного столбца (8 линий) задается комбинацией четырех входных сигналов дешифратора, а выбор строки (8 линий) производится непосредственной подачей сигналов из порта. Такое включение и используется в стенде, фрагмент схемы которого показан на рис. 3.8.

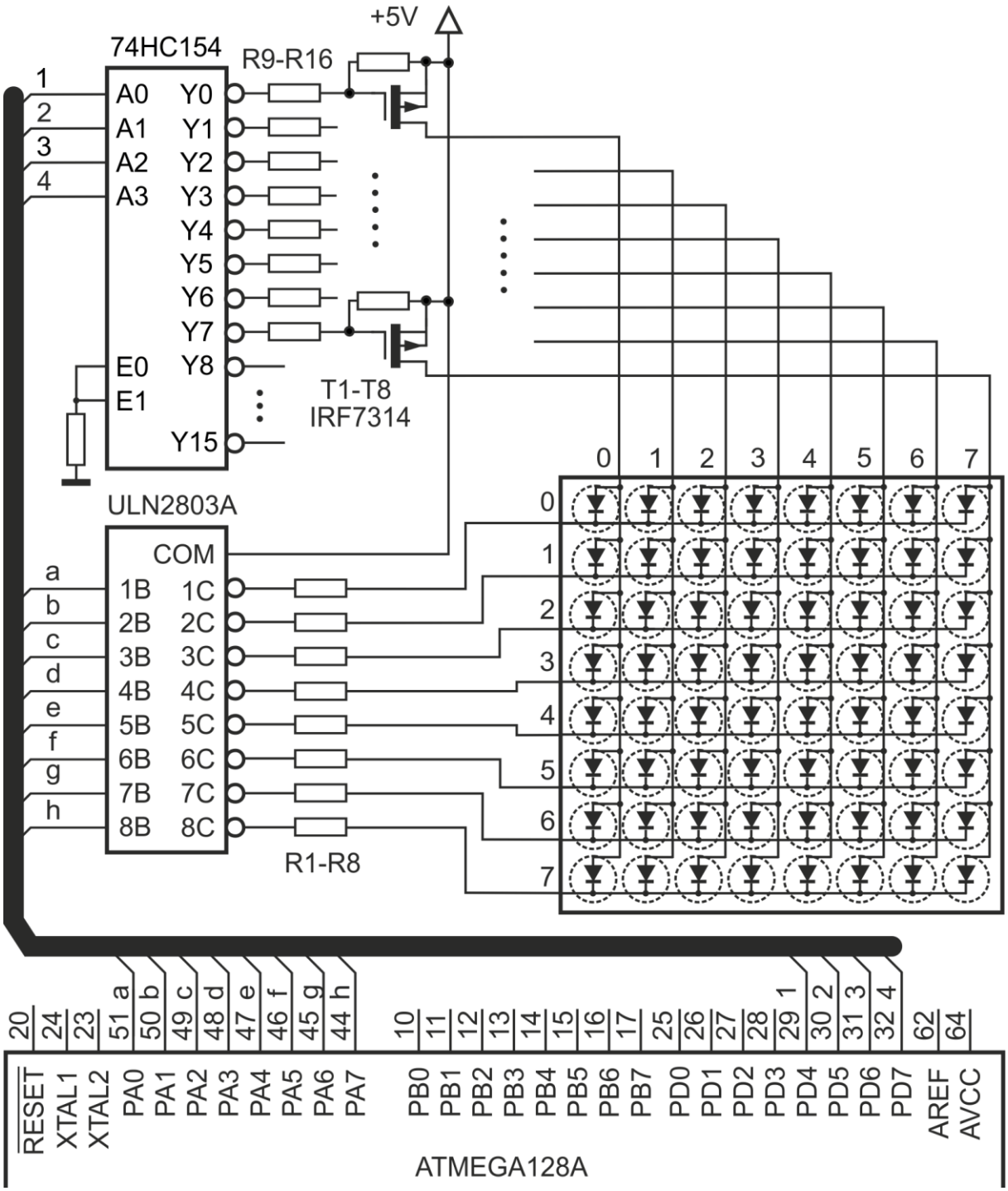


Рис. 3.8. Схема включения части светодиодной матрицы

Поскольку нагрузочная способность разрядов порта для зажигания светодиодов недостаточна, то между ними и линиями столбцов матрицы включены силовые драйверы − микросхемы ULN2803A. При маломощных входных сигналах драйвер позволяет коммутировать на своем выходе значительно более мощную нагрузку. Для линий строк в качестве усилителей использованы мощные полевые транзисторы.

В соответствии с принципом работы дешифратора задание на его входе номера выходной линии делает ее активной, т.е. равной 0 (поскольку выходы дешифратора инверсные). На всех остальных выходных линиях дешифратора устанавливается высокий уровень.

Вследствие этого в любой момент времени в матрице могут гореть светодиоды только одного столбца, номер которого подан на вход дешифратора. Для того чтобы одновременно зажечь несколько светодиодов в столбце, необходимо после активизации столбца вывести в порт PORTA код, в котором зажигаемым точкам соответствует значение 1, а погашенным – 0.

Если смена столбцов с зажженными светодиодами будет меняться чаще, чем 25 раз в секунду, то в силу инерционности зрения будет наблюдаться одновременное свечение светодиодов всех строк, и, соответственно, всей матрицы в целом.

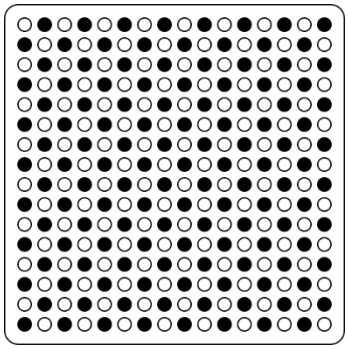
С учетом приведенного описания, приведём на рис. 3.9 для примера алгоритм программы вывода картинки на одном квадрате 8x8 точек светодиодной матрицы.

В начале следует объявить на выход порты МК, задействованные в электрической схеме. В бесконечном цикле исполняется ещё один цикл, на этот раз – конечный, состоящий из восьми итераций по переменной **col**. Одна итерация цикла включает в себя выбор столбца с помощью регистра PORTD, выставление рисунка в столбце с помощью регистра PORTC и выставление временно*й* задержки в одну миллисекунду. После чего, эти инструкции повторяются на следующей итерации.

Подключение всех 4-ёх квадратов приведено на рис. 3.10. Левые два квадрата подключены к порту A МК, правые два – к порту C. Столбцы 0…7 располагаются на верхних квадратах, столбцы 8…15 – на нижних. Выбор столбца (8 вертикальных диодов) осуществляется с помощью порта D на обоих половинах: (0…15 << 4) – номер столбца внутри левой или правой половины. И левая и правая половины матрицы имеют по 16 столбцов.

Промежуточное задание

Написать функцию для вывода 32 столбцов рисунка 16x16 точек. В качестве тестового рисунка можно использовать простейший вариант, когда столбцы состоят из одного из двух байт – либо 0xAA, либо 0x55.



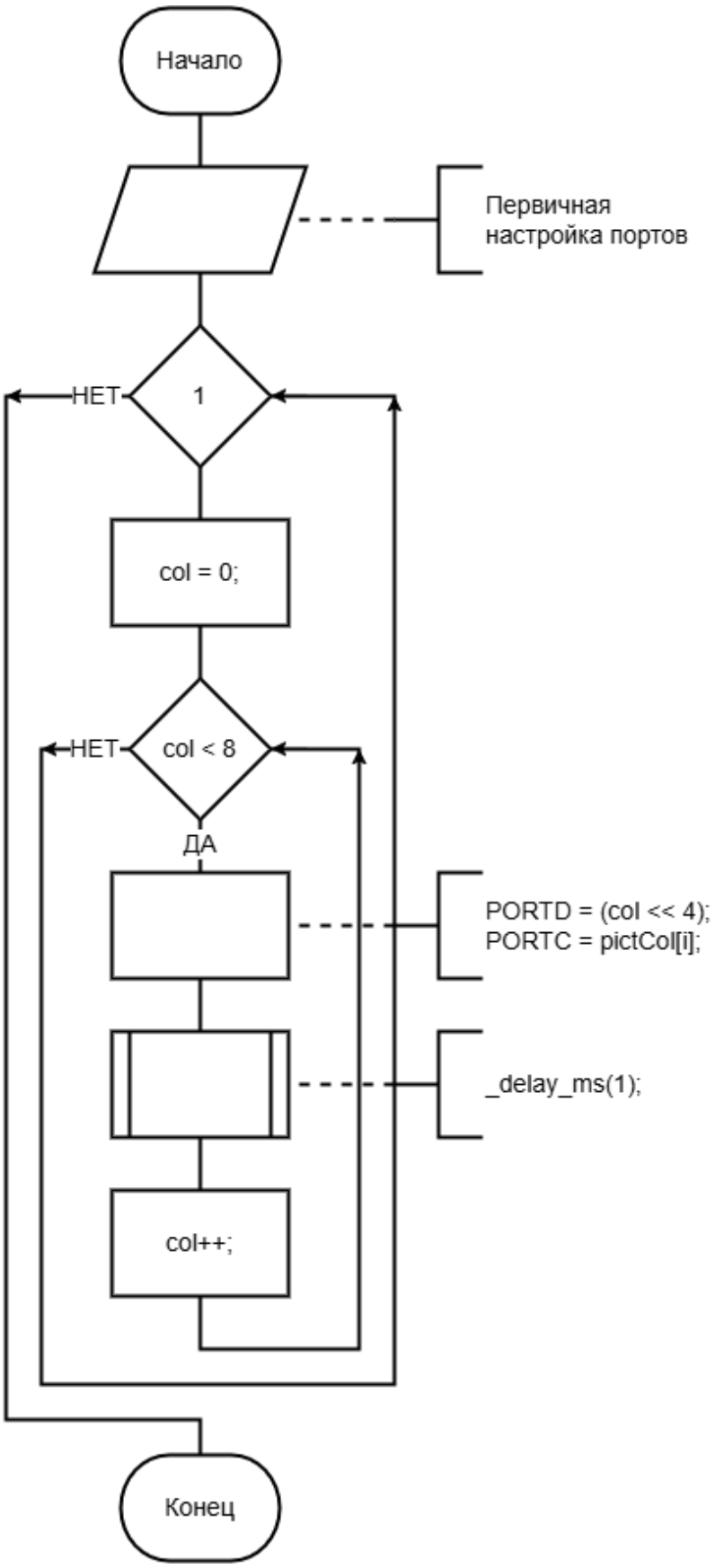


Рис. 3.9. Алгоритм вывода картинки на одном квадрате матрицы

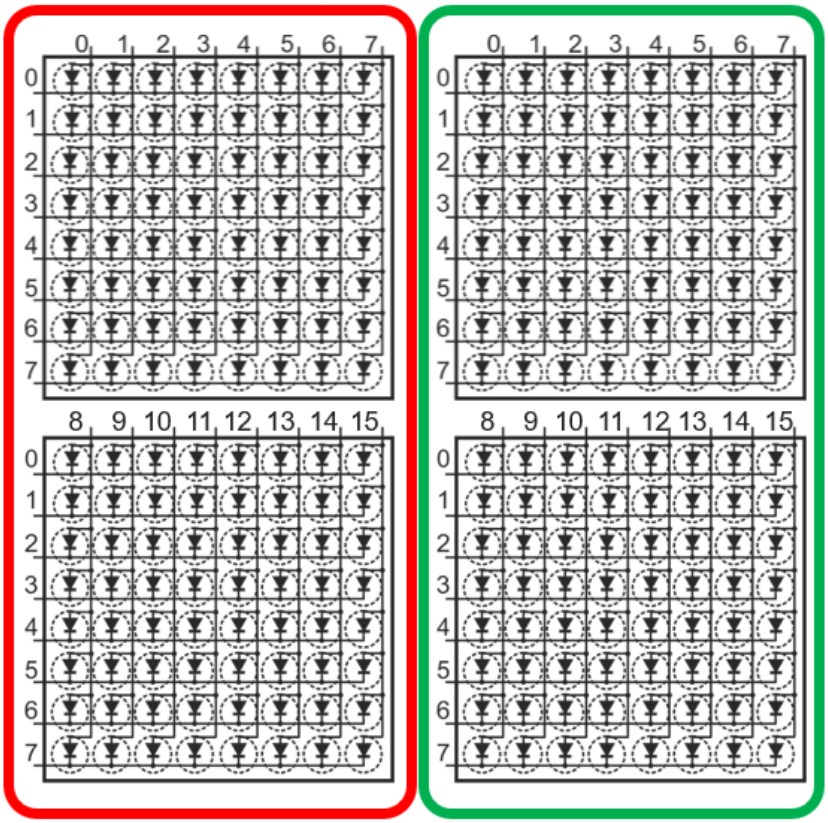


Рис. 3.10. Принцип включения четырёх светодиодных матриц

Методические указания по выполнению части работы с таймерами

Таймеры-счётчики предназначены для точного задания временных интервалов, генерации прямоугольных импульсов и измерения временных характеристик импульсных сигналов.

Микроконтроллер ATmega128A имеет четыре таймера/счётчика: два из них 8-битные (Таймеры №0 и №2) и два – 16-битные (Таймеры №1 и №3). Каждый из таймеров имеет счётный регистр TCNT*n* (где *n* – номер таймера), соответствующего размера, значение в котором автоматически увеличивается с определённой, заранее заданной скоростью. Причём это увеличение происходит не зависимо от работы основной программы. Регистр 8-битного таймера может хранить число из диапазона 0-255, а 16-битного из диапазона 0-65535. При переполнении регистра, его значение становиться равным 0, а также генерируется запрос на прерывание по переполнению, в виде установки флага. Таймер также может генерировать запрос на прерывание по совпадению значения его счётного регистра с заранее заданным числом, хранимым в регистре сравнения OCR*n*, 16-битные таймеры имеют три независимых регистра сравнения (OCR*n*A, OCR*n*B, OCR*n*C, где *n* – номер таймера).

Для базовой настройки любого таймера, следует его включить и выставить предделитель (предделитель определяет значение во сколько раз частота счёта таймера будет меньше частоты ЦП МК). Биты настройки значения предделителя находятся в регистре настройки. Регистры настройки TCCR*n*A и TCCR*n*B 16-битных таймеров приведён ниже (*n* – номер таймера).

Бит	7	6	5	4	3	2	1	0	
	COM*n*A1	COM*n*A0	COM*n*B1	COM*n*B0	COM*n*C1	COM*n*C0	WGM*n*1	WGM*n*0	**TCCR*n*A**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	ICNC*n*	ICES*n*	-	WGM*n*3	WGM*n*2	CS*n*2	CS*n*1	CS*n*0	**TCCR*n*B**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Биты CS*ni* – это биты предделителя. Если необходимо замедлить таймер, пользователь может использовать предделитель. В таблице ниже дано описание битов CS*ni*. Согласно ей, если все биты CS*ni* равны нулю, таймер остановлен, это означает, что, если пользователю нужно запустить таймер, биты CS*ni* должны быть отредактированы. Значения комбинаций битов CS*ni* для 16-битных таймеров приведены в табл. 3.4.

Таблица 3.4

C*n*2	**C*n*1**	**CS*n*0**	**Описание**
0	0	0	Таймер остановлен
0	0	1	Предделитель на 1
0	1	0	Предделитель на 8
0	1	1	Предделитель на 64
1	0	0	Предделитель на 256
1	0	1	Предделитель на 1024

Регистр настройки TCCR*n* 8-битных таймеров приведён ниже (*n* – номер таймера).

Бит	7	6	5	4	3	2	1	0	
	FOC*n*	WGM*n*0	COM*n*1	COM*n*0	WGM*n*1	CS*n*2	CS*n*1	CS*n*0	**TCCR*n***
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Значения комбинаций битов CS*ni* для 8-битных таймеров приведены в табл. 3.5.

Таблица 3.5

CS02	CS01	CS00	Описание	CS22	CS21	CS20	Описание
0	0	0	Таймер остановлен	0	0	0	Таймер остановлен
0	0	1	$f_{CPU}/1$	0	0	1	$f_{CPU}/1$
0	1	0	$f_{CPU}/8$	0	1	0	$f_{CPU}/8$
0	1	1	$f_{CPU}/32$	0	1	1	$f_{CPU}/64$
1	0	0	$f_{CPU}/64$	1	0	0	$f_{CPU}/256$
1	0	1	$f_{CPU}/128$	1	0	1	$f_{CPU}/1024$
1	1	0	$f_{CPU}/256$	1	1	0	Зарезервировано
1	1	1	$f_{CPU}/1024$	1	1	1	Зарезервировано

Разрешение прерываний от таймера происходит через установку битов в регистрах локального разрешения прерываний. Регистр TIMSK отвечает за настройку прерываний от таймеров 0…2, регистр ETIMSK отвечает за настройку прерываний от таймера 3 (и немного за таймер 1).

Бит	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TCIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	**TIMSK**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	-	-	TCIE3	OCIE3A	OCIE3B	TOIE3	OCIE3C	OCIE1C	**ETIMSK**
R/W	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Обозначения: TOIE*n* – бит разрешения прерывания по переполнению таймера *n*, OCIE*nM* – бит разрешения прерывания по совпадению счётного значения таймера TCNT*n* с заранее заданным значением в регистре OCR*n* (также возможен случай – при совпадении со значением регистра OCR*nM*). На данный момент нас интересуют только эти биты.

Условие разрешения прерывания в регистрах TIMSK и ETIMSK является необходимым, но недостаточным, поскольку прерывания должны быть разрешены ещё и глобально. Делается это вызовом специальной машинной команды – `sei();`. Для глобального запрещения прерываний также введена

отдельная команда – **cli();**. Для работы с прерывания необходимо подключить заголовочный файл **#include <avr/interrupt.h>**.

В общем случае запрос на прерывание во время работы МК приводит к временной остановке выполнения основной программы и передаче управления специальной функции-обработчику прерывания (Interrupt Service Routine – ISR), в которой реализуется реакция на событие, вызвавшее запрос прерывания. По окончании работы ISR выполнение основной программы должно возобновляться с того момента, в который она была прервана по запросу прерывания. Необходимость учета особенностей архитектуры МК семейства AVR в программах на языке С привела к введению в стандарт языка расширений, одним из которых является специальное определение ISR в различных средах программирования. В соответствии с этим расширением, ISR записываются следующим образом.

```
ISR (TIMER0_OVF_vect) {
  /* это - функция-обработчик прерывания, которая
  исполнится, когда произойдет событие переполнения
  таймера 0 */
}
ISR (TIMER1_COMPB_vect) {
  /* функция исполнится, когда произойдет событие
  совпадения счётного значения таймера 1 с OCR1B */
}
ISR (TIMER2_COMP_vect) {
  /* функция исполнится, когда произойдет событие
  совпадения счётного значения таймера 2 с OCR2 */
}
```

При работе с прерываниями необходимо помнить некоторые важные моменты:

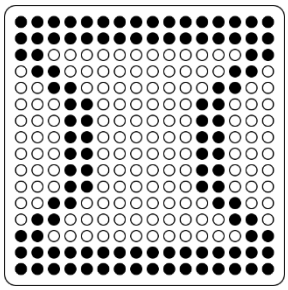
- функции-обработчики прерываний с атрибутом ISR не должны иметь параметров и возвращать значений, поэтому для связи с основной программой используются глобальные переменные;
- каждый источник прерываний имеет только один флаг, указывающий на то, что произошло событие, вызывающее это прерывание, поэтому если до вызова ISR случится несколько событий, обработчик все равно будет вызван только один раз.

Для того чтобы ISR была вызвана необходимо одновременное выполнение трех условий:

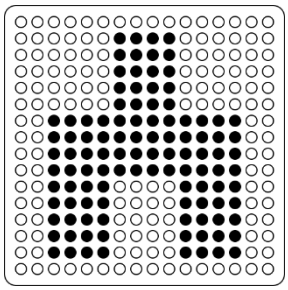
- прерывания должны быть разрешены глобально;
- должно быть разрешено соответствующее прерывание локально внутри периферийного модуля МК;
- должно произойти событие, вызывающее настраиваемое прерывание.

Варианты заданий

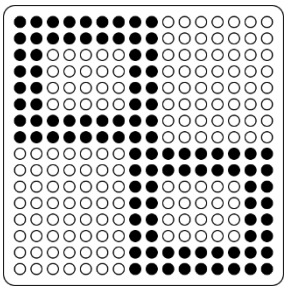
Составить программу вывода на светодиодную матрицу приведенных ниже пиктограмм в соответствии с заданным вариантом работы. Вывод рисунков столбцов следует производить через ISR по любому таймеру/счётчику. ISR рекомендуется настраивать на исполнение раз в 1 миллисекунду. Кроме того, раз в 3 секунды выводимый рисунок инвертируется (инвертируется состояние всех светодиодов). Бесконечный цикл главной функции должен оставаться пустым.



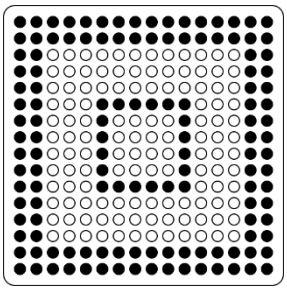
Вариант 1



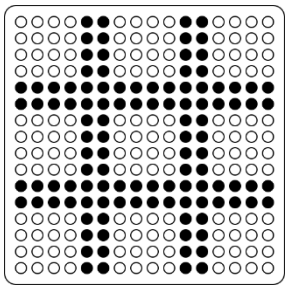
Вариант 2



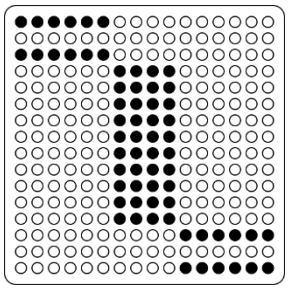
Вариант 3



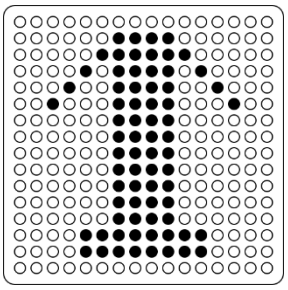
Вариант 4



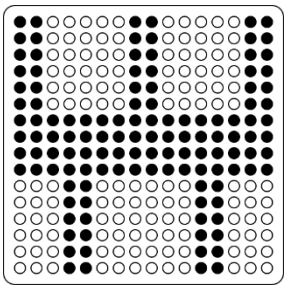
Вариант 5



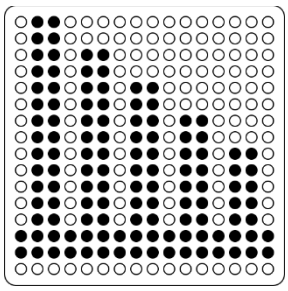
Вариант 6



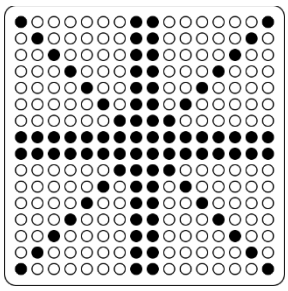
Вариант 7



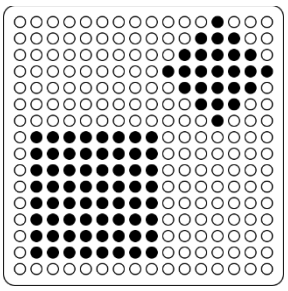
Вариант 8



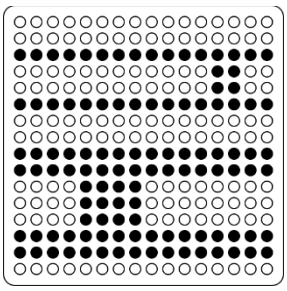
Вариант 9



Вариант 10



Вариант 11



Вариант 12

Проект рекомендуется реализовывать в STUDIO.

ЛР № 3 Организация обмена данными по UART и обработка внешних прерываний

Цель работы. Программирование протокола обмена данными UART и обработка внешних сигналов прерываний (VSCODE).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.11.



Рис. 3.11. Состояние тумблеров конфигурации стенда для работы 3

Методические указания по выполнению части работы с UART

В МК имеется встроенный модуль универсального последовательного интерфейса – UART (более общее название – USART, что расшифровывается как Universal Synchronous/Asynchronous Receiver-Transmitter или Универсальный Синхронно-Асинхронный Приёмо-Передатчик). AVR поддерживает этот протокол в полном объёме полностью аппаратно (на уровне регистров). По структуре протокол UART (для простоты и краткости будем называть его именно так) – это асинхронный последовательный протокол, то есть передающая сторона (master или slave) по очереди выдаёт в линию логический 0 и 1, а принимающая (slave или master, соответственно) отслеживает их и «запоминает». Синхронизация идет по времени – приемник и передатчик заранее договариваются о том, на какой частоте будет идти обмен. Это очень важный момент – если скорость передатчика и приемника не будут совпадать, то передачи может не быть вообще, либо будут считаны не те данные.

Рассмотрим временну*ю* диаграмму протокола передачи UART, приведённую на рис. 3.12.

Вначале передатчик опускает шину данных в низкий уровень – это стартовый бит. После среза на линии связи, приемник выжидает интервал T1 и считывает первый бит, потом через интервалы T2 считываются остальные биты. Последний бит – это стоповый бит, сигнализирующий о том, что передача этого байта завершена. Это простой случай.

В конце байта, перед стоповым битом, может присутствовать бит четности (паритета), который получается если произвести операцию исключающего

ИЛИ (XOR – eXclusive OR) между всеми битами для контроля качества передачи. Также, могут быть два стоповых бита – для надежности. Битов может быть не обязательно 8, а от 5 до 9. Обо всех этих параметрах участники обмена договариваются до начала коммуникации. Общепринятым «шаблоном» является: 8 бит данных, 1 стартовый и стоповый бит и без проверки паритета.



Рис. 3.12. Временная диаграмма протокола передачи данных UART

UART удобно использовать для передачи данных с МК на ПК и обратно. Для этих целей подойдёт, например, модуль преобразования USB-UART, преобразующий UART протокол в импульсы, посылаемые на виртуальный COM-порт ПК.

Для аппаратной организации обмена данными по протоколу UART следует подключить выводы Rx и Tx МК к выводам Tx и Rx модуля UART-USB (именно так – перекрёстно – Rx одного к Tx другого и наоборот, как показано на рис. 3.13).

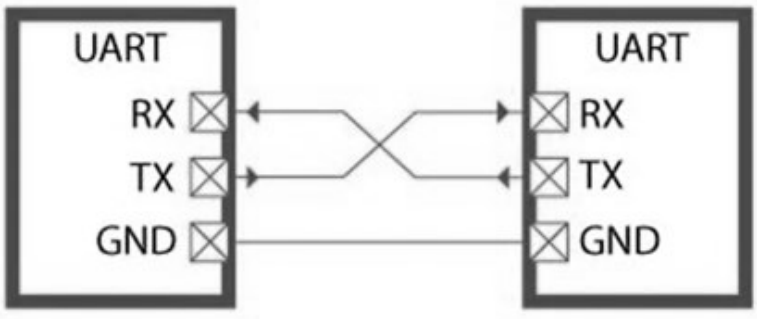


Рис. 3.13. Перекрёстное подключение интерфейсных проводов UART

Обратимся к регистрам МК, ответственным за обмен данными по рассматриваемому протоколу связи.

Буферы принимаемых и передаваемых данных расположены в одном адресном пространстве. При чтении из регистра UDR*n* (*n* – порядковый номер периферийного модуля UART в МК) – данные считываются из буфера, при записи в него – данные записываются в буфер отправки. При отправке пакетов в формате 5, 6, или 7 бит данных – старшие биты игнорируются передатчиком, а при получении – приёмник заполняет их нулями.

Бит	7	6	5	4	3	2	1	0	
	RxB*n* / TxB*n* [7…0]								**UDR*n***
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Настройка UART осуществляется с помощью трёх регистров настройки UCSRA, UCSRB и UCSRC. Начнём с регистра UCSRA.

Бит	7	6	5	4	3	2	1	0	
	RXC*n*	TXC*n*	UDRE*n*	FE*n*	DOR*n*	UPE*n*	U2X*n*	NPCM*n*	**UCSR*n*A**
R/W	R	R/W	R	R	R	R	R/W	R/W	
Н.з.	0	0	1	0	0	0	0	0	

Разряд RXC*n* – флаг завершения приема; выставляется, когда происходит заполнение регистра приемника и очищается, когда буфер приёма пуст. Разряд TXC*n* – флаг завершения передачи; выставляется, когда происходит опустошение регистра передачи. Разряд UDRE*n* – выставляется, когда регистр UDRn передатчика пуст и в него можно записать новый байт.

Запись данных в буфер передачи возможна, только когда установлен флаг UDRE*n* в регистре UCSR*n*A, в противном случае – записанные данные будут проигнорированы передатчиком. После записи в буфер передачи, данные загружаются в передающий сдвиговый регистр и из него поступают на шину TX (пин TxD МК).

Продолжим изучение регистров, на очереди – регистр UCSR*n*B.

Бит	7	6	5	4	3	2	1	0	
	RXCIE*n*	TXCIE*n*	UDRIE*n*	RXEN*n*	TXEN*n*	UCSZ*n*2	RXB8*n*	TXB8*n*	**UCSR*n*B**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Разряд RXEN*n* – разрешение работы приемника. Разряд TXEN*n* – разрешение работы передатчика. Разряд UCSZ*n*2 – формат данных (количество бит в сообщении).

Регистр UCSR*n*C.

Бит	7	6	5	4	3	2	1	0	
	UMSEL*n*1	UMSEL*n*0	UPM*n*1	UPM*n*0	USBS*n*	UCSZ*n*1	UCSZ*n*0	UCPOL*n*	**UCSR*n*C**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	1	1	0	

Разряды UMSEL*n*1:0 - выбор режима работы (00 – синхронный, 01 – асинхронный). Разряды UPM*n*1:0 – выбор режима паритета (00 – отключен, 10 – четность, 11 – нечетность). Разряд USBS*n* – выбор числа стоп-битов (0 – 1 бит, 1 – 2 бита). Разряды UCSZ*n*1:0 – формат данных (вместе с UCSZ*n*2: 000 – 5 бит, 001 – 6 бит, 010 – 7 бит, 011 – 8 бит, 111 – 9 бит).

Скорость обмена данными по UART выбирается из стандартных значений и задаётся с помощью регистров UBRR*n*H и UBRR*n*L.

Бит	7	6	5	4	3	2	1	0	
	-	-	-	-	UBRR*n* [11…8]				**UBRR*n*H**
R/W	R	R	R	R	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	
Бит	7	6	5	4	3	2	1	0	
	UBRR*n* [7…0]								**UBRR*n*L**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Значение в регистры заносится исходя из формул (baud – значение скорости: 2400, 4800, 9600, 14400, 19200 и т.д.):

$$UBRR = \frac{F_{CPU}}{(16 * baud)} \text{ если } \boldsymbol{U2Xn = 0};\ UBRR = \frac{F_CPU}{(8 * baud)} \text{ если } \boldsymbol{U2Xn = 1}.$$

Удобно было бы написать себе немного высокоуровневых функций для работы с UART, а именно – для инициализации (первичной настройки) протокола, а также для передачи данных.

```
  //[]-----------------------------------------------------[]
  //| Назначение: инициализация UART на МК                 |
  //|  включение передатчика, сообщение – 8 бит, 1 стоп,   |
  //|  без проверки паритета, скорость – 19200 бод         |
  //[]-----------------------------------------------------[]
  void uartInit(void) {
    UCSR0B |= (1 << TXEN0); // включение передатчика
    UCSR0C |= (1 << UCSZ01) | (1 << UCSZ00);
    //для значения 19200, UBRR0 = 36 = 0x24
    UBRR0H = 0;
    UBRR0L = 0x24;
}
  //[]-----------------------------------------------------[]
  //| Назначение: передача символа с МК на ПК              |
  //|  ожидание выставления бита UDRE0,                    |
  //|  после чего кладём посылку в UDR0                    |
  //[]-----------------------------------------------------[]
  void uartTransmitByte(char byte) {
    while(!(UCSR0A & (1<<UDRE0)));
    UDR0 = byte;
}
```

Для обмена данными между МК и ПК, на ПК должна быть установлена программа-терминал. Иногда терминал является частью сред разработки, как, например, инструмент Data Visualizer в STUDIO. Терминал не является стандартной утилитой для VSCODE, а устанавливается из галереи расширений и называется Serial Monitor, рассмотрим его.

Чтобы зайти в терминал, достаточно в нижней панели VSCODE перейти на вкладку Serial Monitor (см. рис. 3.14).

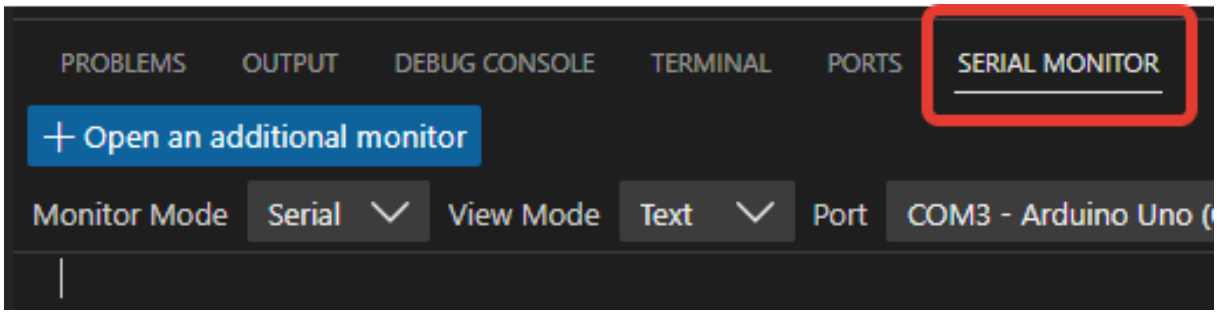


Рис. 3.14. Serial Monitor в VSCODE

Внешний вид окна Serial Monitor приведён на рис. 3.15. Для настройки терминала, следует выбрать COM порт ПК, к которому подключен USB-UART

преобразователь (поле Port, позиция 1 на рис. 3.15), далее настроить скорость передачи (поле Baud rate, позиция 2 на рис. 3.15), после чего нажать на кнопку Start Monitoring / Stop Monitoring (позиция 3 на рис. 3.15). После чего, если МК передаёт какие-либо данные на ПК, они отобразятся в текстовой области. Для передачи текста следует сделать выбор Send as Text (позиция 4 на рис. 3.15)), это признак того, что будет обмен текстовыми байтами.

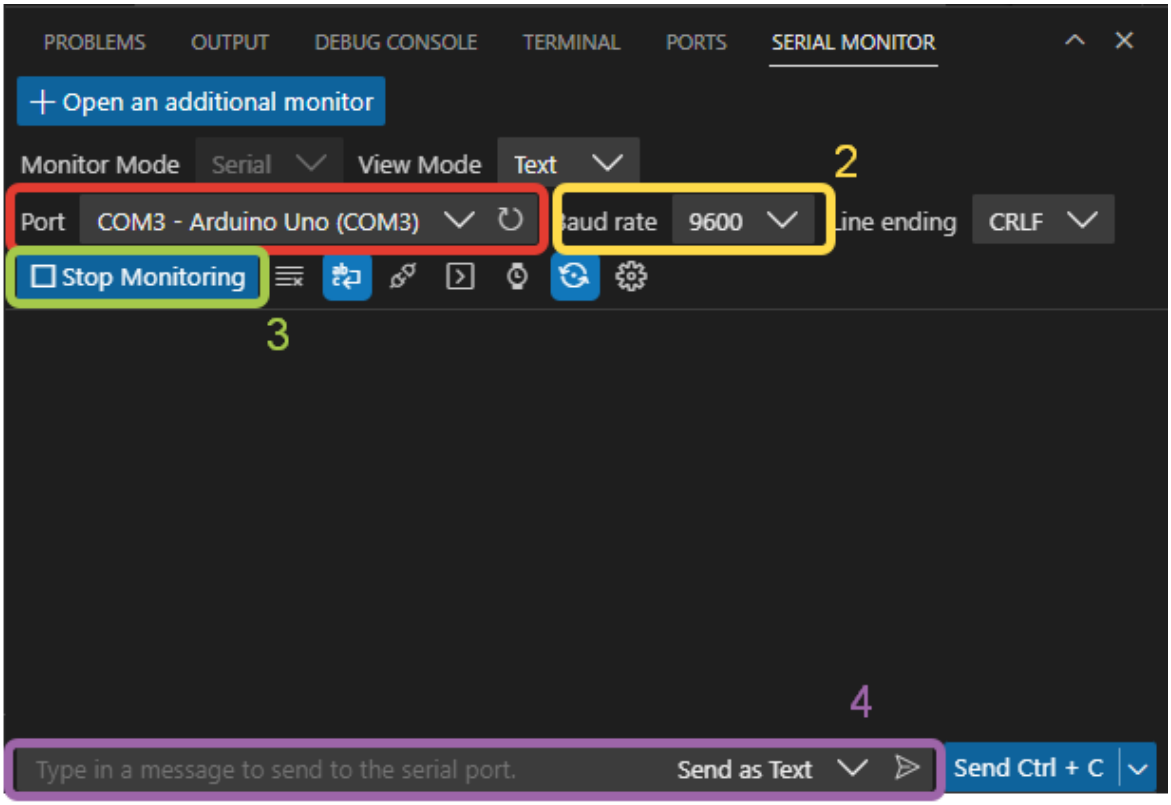


Рис. 3.15. Внешний вид Serial Monitor

Результатом данной части работы должна стать возможность посылать различные числовые и строковые значения с МК в терминал.

Методические указания по выполнению части работы с внешними прерываниями

В системе прерываний МК ATmega128A имеется 34 источника прерываний, восемь из которых являются внешними по входам INT0…INT7. Рассмотрим работу системы прерываний от внешних источников.

Любой запрос на прерывание вызывает установку флага прерывания, соответствующего источнику прерывания, вызвавшего запрос. Для внешних прерываний МК ATmega128A данные флаги расположены в регистре ввода-вывода EIFR.

Бит	7	6	5	4	3	2	1	0	
	INTF7	INTF6	INTF5	INTF4	INTF3	INTF2	INTF1	INTF0	**EIFR**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Выставление данного флага сопровождается переходом программы к выполнению функции-обработчика прерывания ISR, о которой читателю уже известно. Следует только отметить, что вектора прерываний от внешних источников обозначаются INT*i*_vect, таким образом, в качестве примера заголовка функции-обработчика прерывания, можно привести обработчик сигнала на выводе INT4 с функцией **`ISR(INT4_vect)`**.

Как правило, из всех потенциально возможных источников прерывания используются только те, которые необходимы для реализации алгоритма решения поставленной задачи. Поэтому в МК ATmega128A имеется возможность программного запрета (маскирования) прерываний от конкретного источника прерывания в том случае, когда их обработка по каким-либо причинам не требуется. Прерывания от каждого из указанных выше источников могут быть независимо разрешены или запрещены с использованием специального регистра EIMSK.

Бит	7	6	5	4	3	2	1	0	
	INT7	INT6	INT5	INT4	INT3	INT2	INT1	INT0	**EIMSK**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Если в разряд INT0…INT7 регистра EIMSK записать логическую единицу, то работа прерываний по данному источнику разрешена, в противном случае − запрещена.

Если сигнал на INT0…INT7 вызывает прерывание, то соответствующий флаг INTF0…INTF7 принимает состояние логической единицы. Флаг сбрасывается аппаратно после выполнения процедуры обработки прерывания.

Запрос на внешние прерывания возникает при подаче внешних сигналов на выводы INT0…INT7 МК. Эти запросы могут генерироваться по срезу сигнала (1-в-0), фронту сигнала (0-в-1), по низкому логическому уровню, а также по любому изменению уровня на входе (последнее – только для INT4-INT7). Эти установки задаются в специальных регистрах управления внешними прерываниями EICRA (выводы INT3…INT0) и EICRB (выводы

INT7…INT4). Если внешнее прерывание разрешено и настроено на срабатывание при низком уровне сигнала, то прерывание будет инициализироваться постоянно, пока на выводе будет оставаться низкий логический уровень.

Рассмотрим регистр EICRA, отвечающий за настройку типа сигнала внешнего прерывания на выводах МК INT0…INT3.

Бит	7	6	5	4	3	2	1	0	
	ISC31	ISC30	ISC21	ISC20	ISC11	ISC10	ISC01	ISC00	**EICRA**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Пары разрядов ISC0*n*, ISC1*n*, ISC2*n* и ISC3*n* данного регистра являются условиями генерации четырех внешних прерываний INT0…INT3. В табл. 3.6 приведено соответствие задания уровня или фронта для активации прерывания в зависимости от значения битов регистра EICRA.

Таблица 3.6

ISC*n*1	**ISC*n*0**	**Описание**
0	0	Низкий уровень на INT*n* генерирует запрос на прерывание
0	1	Зарезервировано
1	0	Падающий срез на INT*n* генерирует запрос на прерывание
1	1	Нарастающий фронт на INT*n* генерирует запрос на прерывание

Изменять биты рекомендуется при отключенных прерываниях, путем сброса бита разрешения прерывания в регистре EIMSK. После изменения битов ISC*nm*, бит разрешения прерываний можно выставить в единицу. Перед возобновлением работы прерываний необходимо сбросить флаг прерывания путем записи логической единицы во флаг прерывания (INTF*n*) в регистре EIFR.

Рассмотрим регистр EICRB, отвечающий за настройку типа сигнала внешнего прерывания на выводах МК INT4…INT7.

Бит	7	6	5	4	3	2	1	0	
	ISC71	ISC70	ISC61	ISC60	ISC51	ISC50	ISC41	ISC40	**EICRB**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Пары разрядов ISC4*n*, ISC5*n*, ISC6*n* и ISC7*n* данного регистра, являются условиями генерации внешних прерываний INT4…INT7. В табл. 3.7

приведено соответствие задания уровня или фронта для активации прерывания в зависимости от значения битов регистра EICRB.

Таблица 3.7

ISC*n*1	**ISC*n*0**	**Описание**
0	0	Низкий уровень на INT*n* генерирует запрос на прерывание
0	1	Любое изменение логического состояния на INT*n* генерирует запрос на прерывание
1	0	Падающий фронт, выявленный по двум выборкам на INT*n*, генерирует запрос на прерывание.
1	1	Нарастающий фронт, выявленный по двум выборкам на INT*n*, генерирует запрос на прерывание.

Для определения фронтов на выводах INT7…INT4 осуществляется выборка их состояний. Если выбрано прерывание по фронту или изменению уровня, то прерывание будет сгенерировано, если на входе появляется импульс, длительность которого больше одного периода синхронизации. Сделано это для исключения возможности ложного прерывания.

В лабораторной установке запрос прерывания от внешнего источника реализован предельно просто – ко входу INT2 подключена кнопка (поз. 14 на рис. 1.1), с помощью которой формируется сигнал прерывания.

Есть одна особенность, которую необходимо учитывать при разработке программы. Во время работы МК с датчиками, имеющими механические контакты (кнопки, клавиши, клавиатуры и т.д.), возникает явление, называемое дребезгом. Оно заключается в том, что в момент соприкосновения или разъединения контактов сопротивление между ними хаотически меняется в течение некоторого времени, зависящего от материала, конфигурации и состояния контактов, воздействующей на контакты силы и ряда других факторов. Во время дребезга сигнал с контакта может быть прочитан как случайная последовательность нулей и единиц. Подавить это нежелательное явление можно аппаратными средствами, но чаще это делается программным путем.

Наиболее простым способом подавления эффекта дребезга контактов является введение временно*й* задержки. Он основан на том, что программа, обнаружив первое замыкание контакта, запрещает опрос состояния этого контакта на время, заведомо большее длительности времени дребезга. Время

задержки лежит в пределах 10…30 мс и подбирается экспериментально для каждого типа контактов. Повторный опрос состояния клавиши после задержки дает истинное значение этого состояния.

Однако, в функцию-обработчик прерывания вводить явные временны*е* задержки не рекомендуется и для целей подавления дребезга в подавляющем большинстве случаев используют аппаратные таймеры/счётчики МК.

Событие по внешнему сигналу с кнопки запрещает внешние прерывания на данном канале, запускает таймер и разрешает прерывания от него. Прерывание от таймера настраивается на время б*о*льшее времени дребезга. Когда происходит событие таймера первым делом запрещаются прерывания от таймера, разрешённые в ISR по внешнему сигналу, после чего происходит проверка напряжения на выводе МК, к которому подключена кнопка, если кнопка действительно была нажата, то напряжение будет прочитано то, которое должно быть. Если кнопка была нажата, то счётчик нажатий можно инкрементировать, если нет – пропускам этот шаг. В конце ISR таймера следует запретить прерывания от таймера, для избегания ненужных проверок напряжения. Основная функция в этот момент беспрерывно занимается выводом чисел на семисегментные индикаторы и отправкой строк по UART в терминал ПК. Примерный алгоритм работы главной функции и двух функций-обработчиков прерываний приведён на рис. 3.16, а и 3.16, б, соответственно.

Варианты заданий

Во всех вариантах заданий осуществляется внешнее прерывание по входу INT2 (ножка PD2), которое инициируется нажатием подключенной к нему кнопки, дребезг аппаратно гасится. В основной программе производится в бесконечном цикле вывод чисел на семисегментные индикаторы с задержкой 100 мс. По протоколу UART в терминал на ПК должно выводиться текущее количество нажатий на кнопку в виде строки: **Число нажатий: n**. Варианты заданий представлены в табл. 3.8.

Таблица 3.8

№ варианта	Диапазон значений счётчика	Прерывание
1 / 7	100…700, шаг 10	По срезу / По фронту
2 / 8	200…800, шаг 20	По срезу / По фронту
3 / 9	300…900, шаг 30	По срезу / По фронту
4 / 10	400…1000, шаг 30	По срезу / По фронту
5 / 11	500…1100, шаг 20	По срезу / По фронту
6 / 12	600…1200, шаг 10	По срезу / По фронту

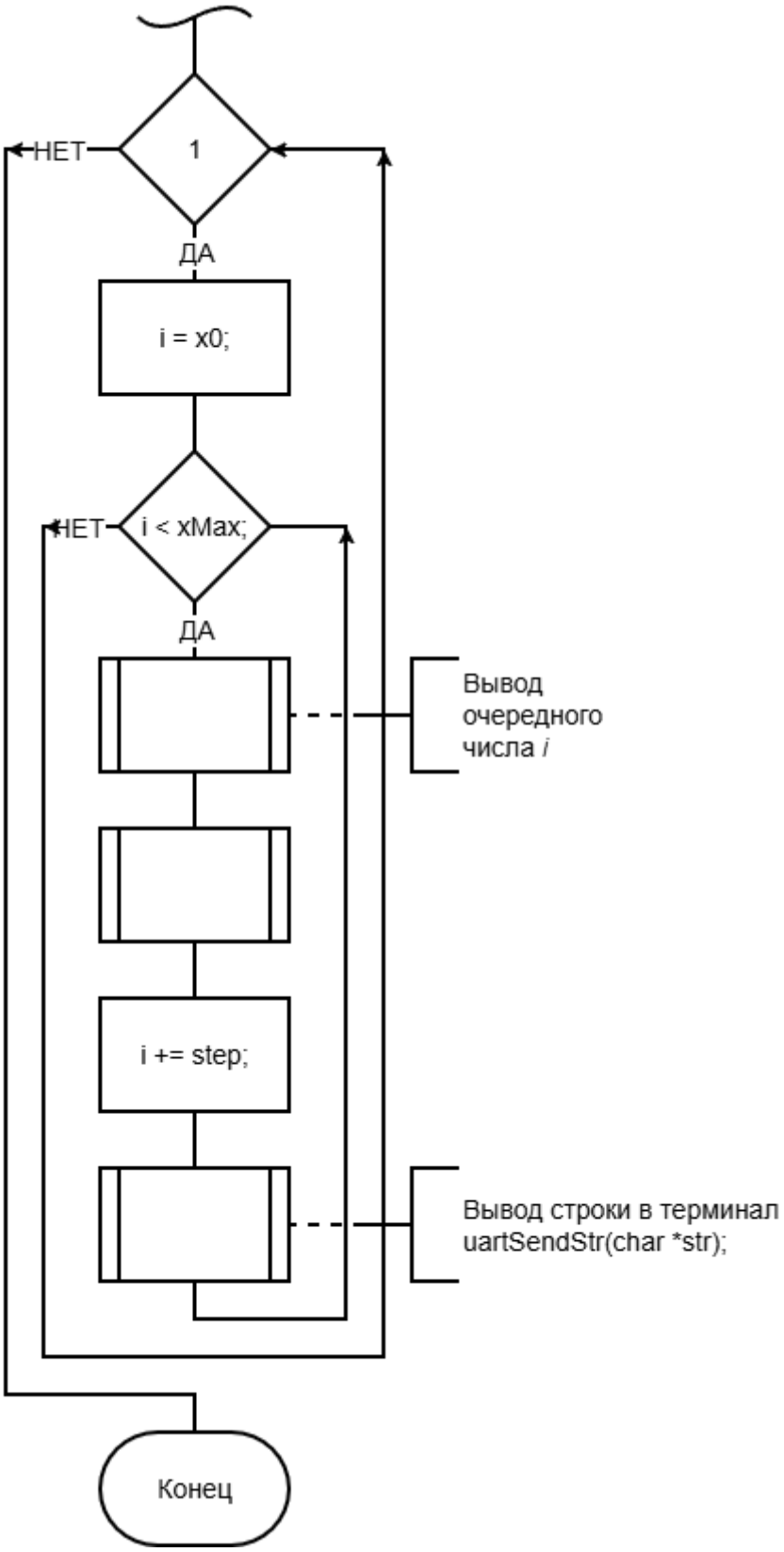


Рис. 3.16, а. Алгоритм функции main()

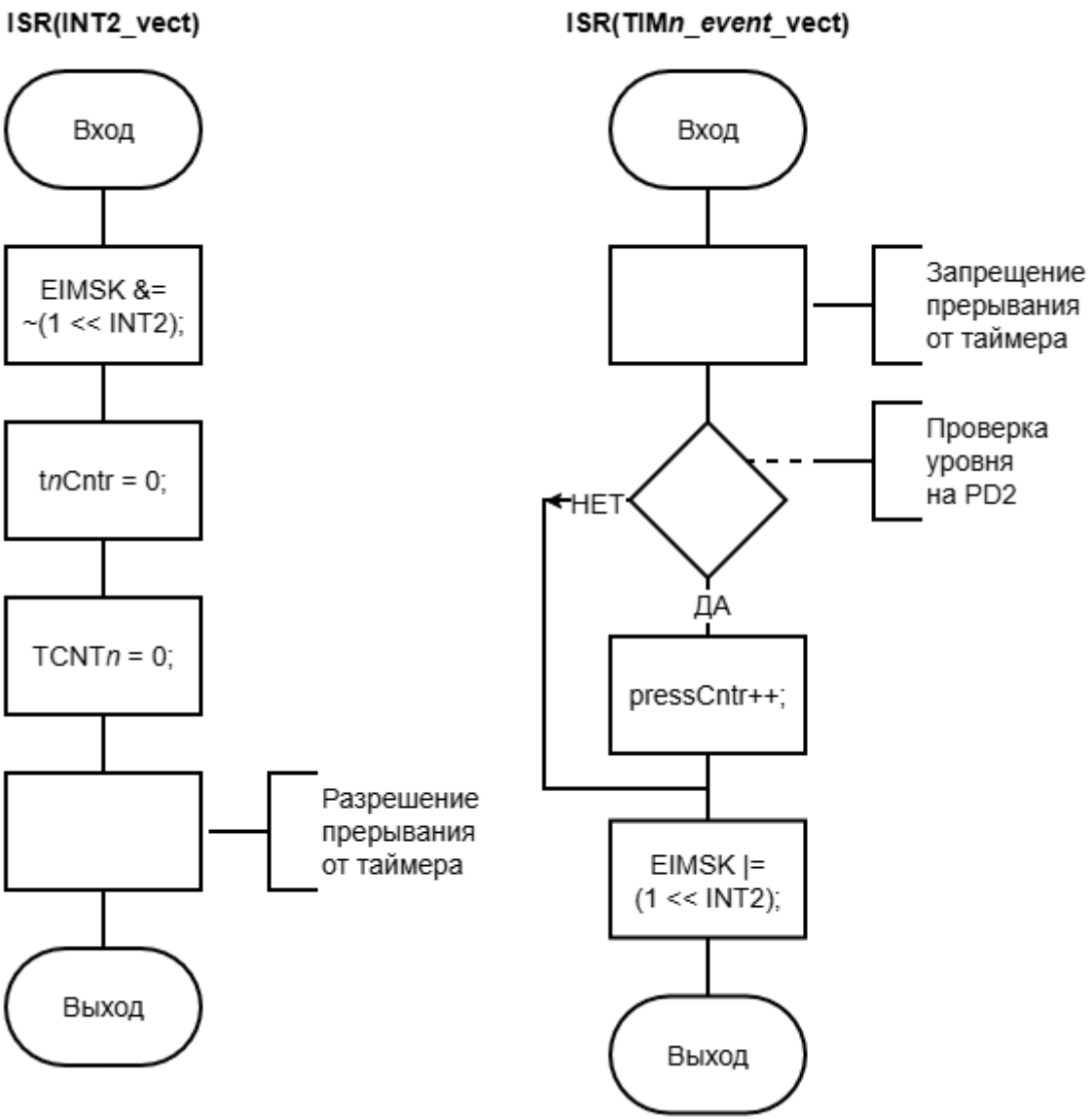


Рис. 3.16, б. Алгоритмы функций-обработчиков прерываний

Проект рекомендуется реализовывать в VSCODE.

ЛР № 4 Управление шаговым двигателем с использованием таймеров

Цель работы. Изучение принципов программного управления шаговым двигателем с использованием таймеров/счётчиков МК (VSCODE).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.17.



Рис. 3.17. Состояние тумблеров конфигурации стенда для работы 4

Методические указания по выполнению работы

Шаговый двигатель (ШД) является одним из широко используемых элементов современных систем автоматического управления. Он представляет собой бесколлекторный двигатель постоянного тока с фиксированными положениями вала. Просто подавая на двигатель питание, заставить его работать невозможно. Для его работы необходимо подавать на обмотки двигателя специальным образом сформированные последовательности импульсов, каждый из которых вызывает перемещение вала двигателя на заданный угол. Конструкция шагового двигателя и схема включения его обмоток показаны на рис. 3.18.

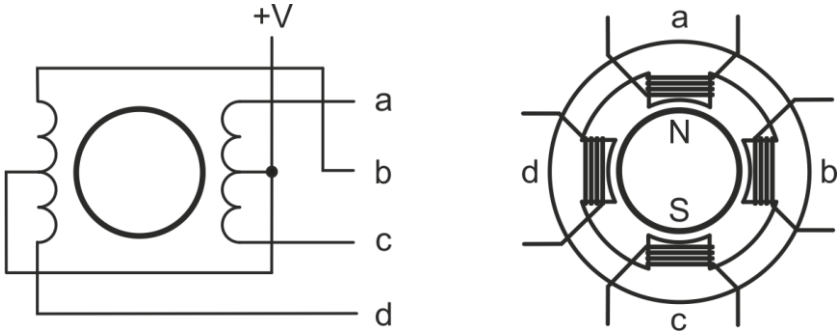


Рис. 3.18. Конструкция и схема включения обмоток ШД

В ШД управляющие импульсы подаются поочередно на обмотки статора, в результате чего ориентация магнитного потока в пространстве между полюсами статора дискретно меняется и вызывает поворот ротора на некоторый фиксированный угол, называемый шагом ротора. Для того чтобы заставить вал двигателя вращаться, необходимо поочередно подавать напряжение на обмотки статора (фазы) в нужной последовательности. Смена

направления вращения вала осуществляется простой заменой последовательности подключения обмоток на обратную, а изменение частоты вращения вала − изменением длительности управляющих импульсов.

В лабораторном стенде используется униполярный ШД с четырьмя обмотками, который имеет шаг 1.8º и, вследствие этого, для полного оборота вала необходимо подать на обмотки 200 импульсов.

Существует несколько способов управления ШД, каждый из которых может быть достаточно просто реализован программно с использованием управляющего МК. Рассмотрим эти способы управления более подробно.

Первый способ управления обеспечивается последовательной коммутацией фаз, при которой в каждый момент времени включена только одна обмотка. Этот способ управления называют однофазным полношаговым режимом. Схема коммутации обмоток ШД для однофазного полношагового режима показана на рис. 3.19.

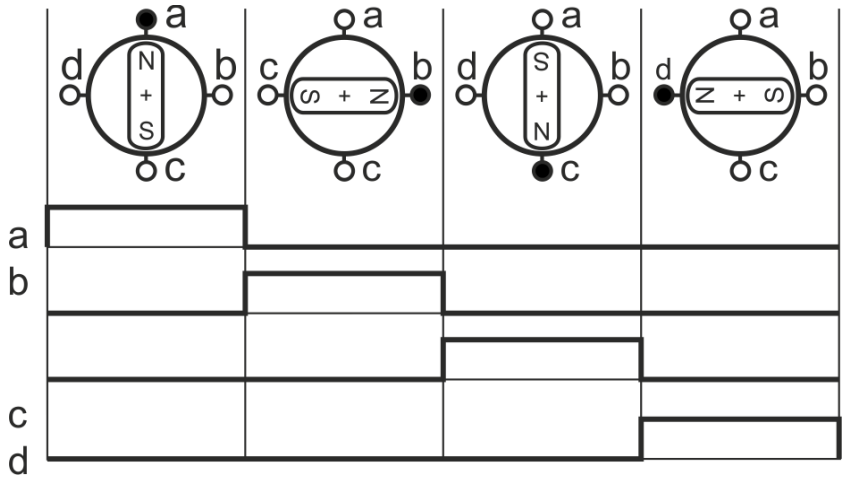


Рис. 3.19. Схема коммутации обмоток ШД для однофазного полношагового режима

Во втором способе коммутация фаз производится с перекрытием: в каждый момент времени включены две соседние обмотки двигателя и поэтому его называют двухфазным полношаговым режимом. Этот способ управления обеспечивает такой же угол шага, как и первый способ, но положение точек равновесия ротора смещено на полшага и ротор фиксируется в промежуточных позициях между полюсами статора. Кроме того, при этом способе управления обеспечивается больший момент, чем в случае одной включенной фазы. Схема коммутации обмоток ШД для реализации двухфазного полношагового режима показана на рис. 3.20.

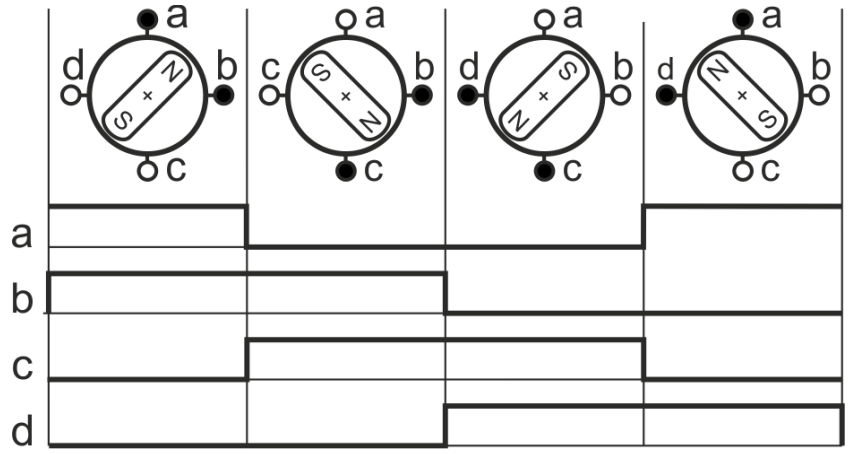


Рис. 3.20. Схема коммутации обмоток ШД для двухфазного полношагового режима

Рассмотренные способы являются простейшими способами управления ШД со всеми присущими им недостатками: малым числом шагов на оборот, повышенной шумности, колебаниями вала при перемещении от шага к шагу. От этих недостатков избавляют более сложные алгоритмы управления, к которым можно отнести следующий способ управления.

Этот способ является комбинацией первых двух и называется полушаговым режимом, при котором двигатель делает шаг, равный половине основного. Каждый второй шаг запитана лишь одна фаза двигателя, а в остальных случаях – две. В результате угловое перемещение ротора составляет половину угла шага для первых двух способов управления. Схема коммутации обмоток ШД для полушагового режима показана на рис. 3.21.

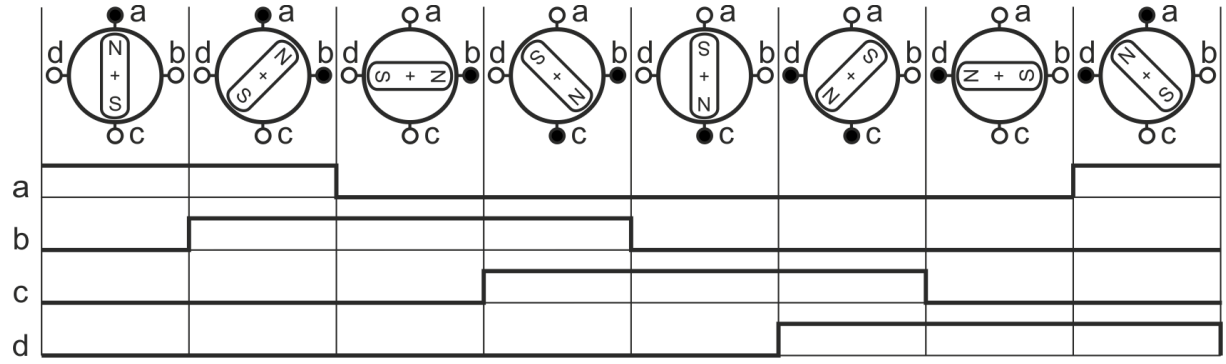


Рис. 3.21. Схема коммутации обмоток ШД для полушагового режима

Для вращения вала двигателя необходимо подавать импульсы напряжения на каждую из четырех обмоток статора в той последовательности, которая определяется используемым способом управления. Вал двигателя при этом будет на каждом шаге поворачиваться на фиксированный угол, величина которого определяется конструкцией двигателя и составляет обычно единицы

градусов. Скорость вращения вала двигателя можно изменять введением пауз различной длительности между моментами переключения фаз.

Схема подключения ШД к МК показана на рис. 3.22.

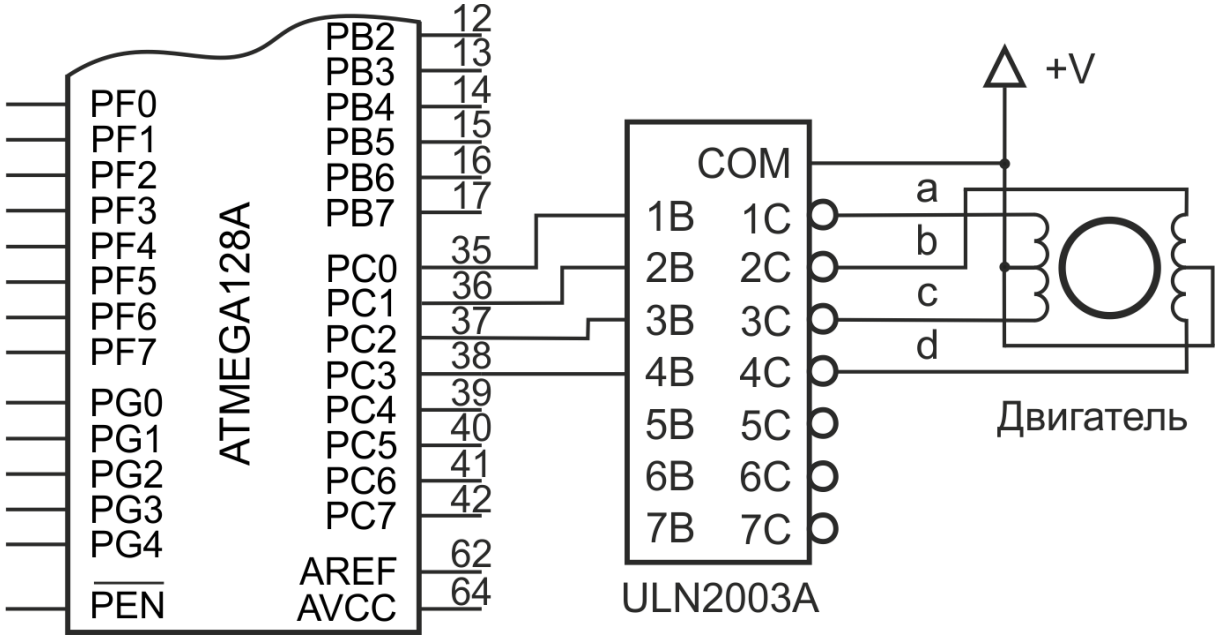


Рис. 3.22. Схема подключения шагового двигателя

Поскольку при работе ШД через его обмотки протекает ток, значительно превышающий максимально допустимый ток порта МК, то для подключения двигателя используется уже знакомая нам силовая микросхема – драйвер ULN2003A, управляемый логическими сигналами с разрядов порта.

Как следует из схемы, для подачи напряжения на обмотку ШД необходимо занести в соответствующий разряд порта PORTC значение логической единицы. В этом случае на выходе драйвера появляется потенциал низкого логического уровня, и в подключенной к нему обмотке формируется передний фронт управляющего импульса. При занесении в разряд PORTC логического нуля на соответствующей обмотке формируется задний срез импульса. Длительность импульса задается введением заданной задержки между этими двумя действиями.

В качестве примера приведем фрагмент программы, реализующей выполнение одного шага в однофазном полношаговом режиме управления двигателем.

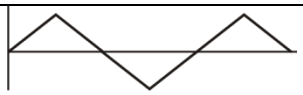
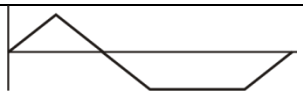
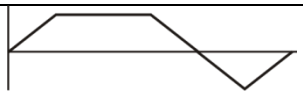
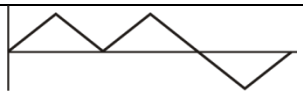
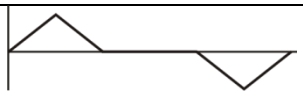
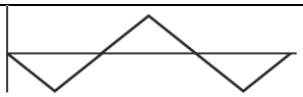
```
DDRC = 0x0F;              // младшие разряды PORTC - выход
PORTC |= (1 << 0);        // сформировать фронт импульса
_delay_ms(20);            // задать длительность импульса
PORTC &= ~(1 << 0);       // сформировать срез импульса
```

Очевидно, что скорость вращения вала двигателя можно изменять заданием длительности управляющих импульсов. При выполнении заданий следует учитывать, что предельная скорость вращения ограничена конструкцией используемого ШД и не превышает 150…200 оборотов в минуту в зависимости от режима управления.

Варианты заданий

В соответствии с вариантом задания, представленном в табл. 3.9, составьте программу управления шаговым двигателем для реализации заданного режима управления и графика изменения скорости вращения.

Таблица 3.9

Вариант задания	График изменения скорости вращения	Режим управления
1		Двухфазный полношаговый
2		Полушаговый
3		Двухфазный полношаговый
4		Полушаговый
5		Двухфазный полношаговый
6		Полушаговый
7		Двухфазный полношаговый
8		Полушаговый
9		Двухфазный полношаговый
10		Полушаговый
11		Двухфазный полношаговый
12		Полушаговый

Пошаговое руководство по выполнению работы:

1. установить значение минимальной и максимальной «скорости» ШД с использованием функции задержки и без использования таймера;
2. настроить подходящий таймер/счётчик таким образом, чтобы период переключения катушек ШД, установленный на предыдущем шаге, укладывался в диапазон счёта таймера;
3. настроить прерывание от таймера/счётчика и написать соответствующую ISR, внутри которой будет происходить перемещение вала мотора на один шаг, согласно режиму управления;
4. написать функцию, реализующую любой тип прямой линии графика изменения скорости вращения ШД из поставленного задания.

Проект рекомендуется реализовывать в VSCODE.

ЛР № 5 Разработка устройства «Калькулятор» на базе с использованием символьного дисплея и матричной клавиатуры

Цель работы. Программирование процедур вывода символьной информации на жидкокристаллический индикатор (ЖКИ) и ввода информации с использованием клавиатуры (VSCODE).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.23.



Рис. 3.23. Состояние тумблеров конфигурации стенда для работы 5

Методические указания по выполнению части работы с ЖКИ

Алфавитно-цифровые жидкокристаллические индикаторы представляют собой недорогое и удобное решение, позволяющее обеспечить отображение большого объема информации при хорошей различимости и низком энергопотреблении.

Управление ЖКИ требует выполнения целого ряда весьма специфических действий, таких, например, как генерация управляющих напряжений для адресации множества ячеек, формирование символов и курсора, и т. п. Для аппаратной реализации таких действий выпускаются специализированные микросхемы – контроллеры ЖКИ, которые берут на себя всю низкоуровневую часть реализации индикатора. Промышленным стандартом такой микросхемы, применяемой для построения большинства матричных знакосинтезирующих ЖКИ, является контроллер HD44780 фирмы Hitachi. Этот встроенный непосредственно в ЖКИ контроллер подключается к целевому МК через параллельный интерфейс и предоставляет программисту ряд высокоуровневых команд (например, вывод символа в указанной позиции экрана, сдвиг информации на экране или отображение мигающего курсора). Контроллер также содержит встроенный знакогенератор и экранную память, избавляя разработчика от необходимости рисовать собственные шрифты.

Основными элементами контроллера ЖКИ, с которыми приходится взаимодействовать пользователю при программном управлении, являются регистр данных DR (Data Register), регистр команд IR (Instruction Register),

видеопамять DDRAM (Display Data RAM), оперативное запоминающее устройство знакогенератора CGRAM (Character Generator RAM), счетчик адреса памяти AC (Address Counter) и флаг занятости контроллера BF (Busy Flag).

ASCII-коды отображаемых символов через регистр данных DR помещаются в видеопамять DDRAM по текущему адресу, указываемому счетчиком адреса AC. Видеопамять, имеющая общий объем 80 байт, предназначена для хранения кодов символов, отображаемых на экране ЖКИ. Видеопамять организована в виде двух буферов по 40 символов в каждом, и чаще всего длина буфера превышает число символов, выводимых в строке индикатора. В этом случае отображаются только те символы из DDRAM, которые попадают в окно индикатора, как это показано на рис. 3.24.



Рис. 3.24. Отображение символов из DDRAM в окне индикатора

В системе команд ЖКИ имеются команды, задающие смещение окна и направление этого смещения. Смещая окно индикатора относительно буфера данных, можно отображать в окне различные области буфера. Сдвиг окна индикатора относительно буфера данных для верхней и нижней строк происходит синхронно. Курсор будет виден на индикаторе только в том случае, если он попал в зону видимости DDRAM.

При внимательном рассмотрении рисунка можно заметить важную особенность. Первая строка DDRAM содержит 40 ячеек, которые адресуется от 0 до 39 (0x27), а вот 40 ячеек второй строки заданы адресами от 64 (0x40) до 103 (0x67). Значения адресов DDRAM в диапазоне 40…63 (0x28...0x3F) являются неопределенными и результаты работы с ними могут быть непредсказуемыми.

Информация, попадающая в регистр команд IR, интерпретируется контроллером ЖКИ как управляющая последовательность. Чтение регистра IR возвращает в семи младших разрядах текущее значение счетчика AC, а в старшем разряде – флаг занятости BF.

Необходимо учитывать, что большинство операций, выполняемых контроллером, занимают время около 40 мкс, а время выполнения некоторых

доходит до единиц миллисекунд. Индикатором окончания любой операции служит сброс флага занятости BF, поэтому для определения момента окончания операции программа может опрашивать флаг BF. Второй, более простой и чаще используемый способ заключается в том, что программист, зная, сколько времени требуется на обработку той или иной команды, просто выполняет в программе заведомо достаточную для выполнения команды временную задержку после каждой передачи информации в ЖКИ.

Для реализации интерфейса МК с контроллером HD44780 используется 11 информационных линий, из которых восемь (DB0…DB7) служат шиной для передачи кодов отображаемых символов или команд, а на три линии (RS, R/W, E) подаются управляющие сигналы.

При помощи сигнала на линии RS (Register Select) МК сообщает контроллеру индикатора, какой регистр – IR или DR, должен быть выбран и что будет передаваться по шине – команда или данные. Потенциал на управляющем выводе R/W задает направление передачи данных: запись в память индикатора (R/W = 0) или считывание из памяти (R/W = 1). Сигнал на линии E (Enable) является стробом, сопровождающим передачу команд и данных. Запись информации в ЖКИ происходит по переходу этого сигнала из 1 в 0.

Список всех управляющих комбинаций сигналов RS, R/W и разрядов регистра IR, а также выполняемые по ним команды приведены в табл. 3.810.

Таблица 3.10

RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0	Назначение команды
0	0	0	0	0	0	0	0	0	1	Очистить дисплей и установить курсор в нулевую позицию (адрес 0)
0	0	0	0	0	0	0	0	1	*	Установить курсор в нулевую позицию (адрес 0). Установить дисплей относительно буфера DDRAM в начальную позицию.
0	0	0	0	0	0	0	1	I/D	S	Установить направление сдвига курсора влево или вправо (I/D = 0/1) при записи очередного кода в DDRAM. Разрешить (S = 1) сдвиг окна вместе со сдвигом курсора
0	0	0	0	0	0	1	D	C	B	Включить-выключить (D = 1/0) индикатор. Зажечь-погасить (C = 1/0) курсор. Сделать курсор в виде мигающего знакоместа (B = 1)
0	0	0	0	0	1	S/C	R/L	*	*	Переместить курсор (S/C = 0) или сдвинуть дисплей (S/C = 1) вправо (R/L = 1) или влево (R/L = 0)

Продолжение Таблицы 3.10

0	0	0	0	1	DL	N	F	*	*	Установить разрядность шины данных 4 или 8 бит (DL = 0/1), количество строк – одна или две (N = 0/1), шрифт – 5 × 7 или 5 × 10 точек (F = 0/1)
0	0	0	1	AG	AG	AG	AG	AG	AG	Установка адреса CGRAM.
0	0	1	AD	AD	AD	AD	AD	AD	AD	Установка адреса DDRAM.
0	1	BF	AC	AC	AC	AC	AC	AC	AC	Чтение busy-флага BF и счетчика адреса AC
1	0	Данные								Запись данных в DDRAM или CGRAM
1	1	Данные								Чтение данных из DDRAM или CGRAM

В лабораторном стенде используется ЖКИ WH1602C-YGH фирмы Winstar, имеющий две строки по 16 символов в каждой. Схема подключения ЖКИ к МК показана на рис. 3.25.

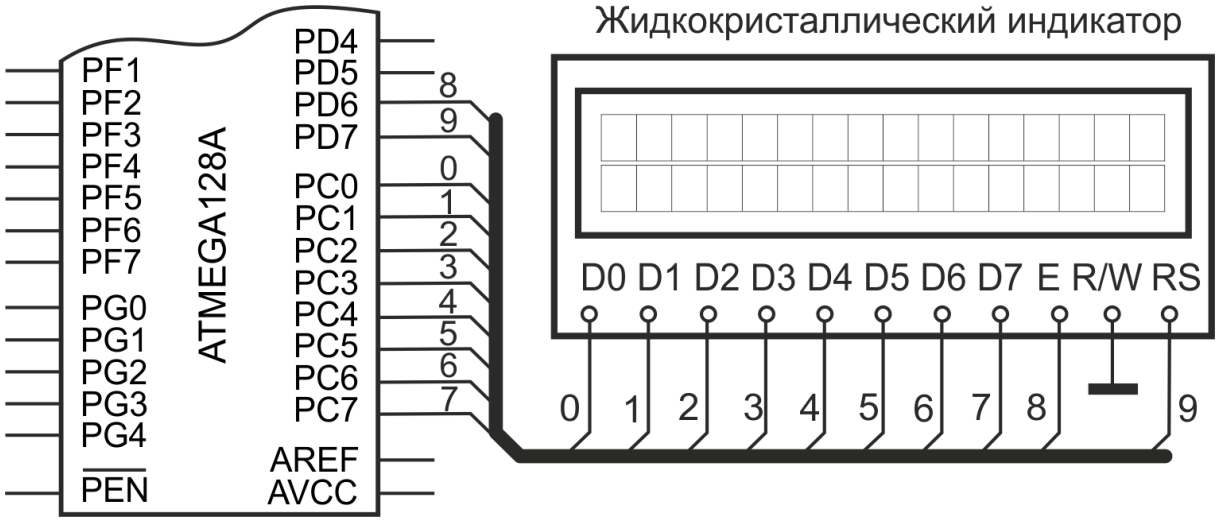


Рис. 3.25. Схема подключения ЖКИ к МК

После включения питания контроллер ЖКИ осуществляет начальную инициализацию индикатора, во время которой по умолчанию производится очистка окна и установка следующих параметров (см. табл. 3.10):

- DL = 1 – разрядность шины данных равна 8;
- N = 0 – однострочный дисплей;
- F = 0 – размер знакоместа 5 × 7 точек;
- B = 0 – мерцание курсора отключено;
- C = 0 – курсор выключен;
- D = 0 – индикатор выключен;
- I/D = 1 – сдвиг курсора вправо;
- S = 0 – сдвиг окна отключен.

Естественно, что такой набор параметров в редком случае может быть признан подходящим (например, в нашем случае используется двустрочный дисплей). Кроме того, инициализация будет выполнена корректно, если напряжение источника питания устанавливает номинальное напряжение менее, чем за 10 мс. Поэтому перед использованием рекомендуется провести повторную программную инициализацию индикатора с установкой необходимых параметров.

При программной инициализации индикатора для работы с 8-разрядной шиной производителем контроллера дисплея рекомендуется выполнять следующую последовательность действий.

После подачи напряжения питания необходимо выдержать паузу более 15 мс для установления номинального напряжения, но из-за различий характеристик используемых источников питания лучше задать длительность паузы в 0.1…0.2 с. Первой операцией задать RS = 0, R/W = 0 и послать команду 0x30, затем сделать задержку не менее 4.1 мс и повторить выполнение команды 0x30. На следующем шаге необходимо вновь выдержать паузу, на этот раз более 100 мкс, и в третий раз повторить команду 0x30. Эти три операции являются инициализирующими, и следом за ними выполняются команды задания режимов работы. После выполнения этих команд необходимо проверять состояние флага занятости BF, но проще выдержать паузу в 100…200 мкс и затем выдавать следующую инструкцию.

Дальнейшая последовательность команд задания режимов для индикатора лабораторного стенда такова:

1. Пауза 100…200 мкс и передача кода 0x38

RS	R/W
0	0

DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	1	DL	N	F	0	0

DL = 1 (8-битный интерфейс); N = 1 (2 строки); F = 0 (символы 5 × 7 точек).

2. Пауза 100…200 мкс и передача кода 0x0E

RS	R/W
0	0

DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	1	D	C	B

D = 1 (дисплей вкл.); C = 1 (курсор вкл.); B = 0 (мерцание выкл.).

3. Пауза 100…200 мкс и передача кода 0x06

RS	R/W
0	0

DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	1	I/D	S

I/D = 1 (сдвиг курсора вправо); S = 0 (нет сдвига дисплея).

Поскольку процедура инициализации ЖКИ представляет собой последовательность команд, то полезным будет предварительное создание функции записи команды в регистр команд ЖКИ

```
#define RS    7          // выбор регистра
#define E     6          // строб передачи

void lcdCmd(uint8_t cmd) {
 //[]-----------------------------------------------[]
 //| Назначение: запись кодов в регистр команд ЖКИ  |
 //| Входные параметры: cmd - код команды           |
 //[]-----------------------------------------------[]
 DDRC = 0xFF;       // все разряды PORTC на выход
 DDRD |= ((1<<E)|(1<<RS)); // разряды PORTD на выход
 PORTD &= ~(1<<RS);    // выбор регистра команд RS=0
 PORTC = cmd;       // записать команду в порт PORTC
 PORTD |= (1<<E); // \ сформировать на
 _delay_us(5);      // | выводе E строб 1-0
 PORTD &= ~(1<<E);// / передачи команды
 _delay_ms(5);      // задержка для завершения записи
}
```

С учетом этого можно реализовать процедуру инициализации ЖКИ в виде функции.

```
void lcdInit(void) {
 //[]-----------------------------------------------[]
 //| Назначение: инициализация ЖКИ                  |
 //[]-----------------------------------------------[]
 DDRC = 0xFF;           // все разряды PORTC на выход
 DDRD |= ((1<<E)|(1<<RS)); // разряды PORTD на выход
 _delay_ms (100); // задержка для установления напряжения питания
 lcdCmd (0x30);    // \ вывод
 lcdCmd (0x30);    // | трех
 lcdCmd (0x30);    // / команд 0x30
 lcdCmd (0x38);    // 8 разр.шина, 2 строки, 5 × 7 точек
 lcdCmd (0x0E);    // включить ЖКИ и курсор, без мерцания
 lcdCmd (0x06);    // инкремент курсора, без сдвига экрана
 lcdCmd (0x01);    // очистить экран, курсор в начало
}
```

Используемый в лабораторном стенде индикатор русифицирован, однако коды символов кириллицы в ЖКИ не совпадают со стандартными кодами из

таблицы символов ASCII и перед выводом на ЖКИ символов кириллицы необходимо производить их перекодировку. Не вдаваясь в подробности этой операции, представим функцию, реализующую процедуру такой перекодировки.

```
uint8_t  tabCon[]  ={0x41,0xA0,0x42,0xA1,0xE0,0x45,0xA3,0xA4,
0xA5,0xA6,0x4B,0xA7,0x4D,0x48,0x4F,0xA8,0x50,0x43,0x54,0xA9,
0xAA,0x58,0xE1,0xAB,0xAC,0xE2,0xAD,0xAE,0x62,0xAF,0xB0,0xB1,
0x61,0xB2,0xB3,0xB4,0xE3,0x65,0xB6,0xB7,0xB8,0xB9,0xBA,0xBB,
0xBC,0xBD,0x6F,0xBE,0x70,0x63,0xBF,0x79,0x5C,0x78,0xE5,0xC0,
0xC1,0xE6,0xC2,0xC3,0xC4,0xC5,0xC6,0xC7};

uint8_t code(uint8_t symb) {
 //[]-----------------------------------------------[]
 //| Назначение: перекодировка символов кириллицы     |
 //| Входные параметры: symb – символ ASCII           |
 //| Функция возвращает код отображения символа       |
 //[]-----------------------------------------------[]
 return (symb >= 192 ? TabCon[symb-192]: symb);
}
```

Методические указания по выполнению части работы с клавиатурой

Одним из широко распространенных в системах управления и контроля устройств, предназначенных для ввода информации в МК, является матричная клавиатура. В стенде используется пленочная клавиатура с организацией в виде матрицы 4 × 3. Схема подключения клавиатуры к МК показана на рис. 3.26.

Один вывод каждой клавиши соединяется с вертикальной линией сканирования, а второй вывод – с горизонтальной линией возврата. Обнаружение нажатой клавиши и ее последующая идентификация основаны на использовании процедуры сканирования, которая сводится к поочередной установке каждой из линий сканирования в состояние логической единицы (при одновременном обнулении остальных) и опросу состояния линий возврата.

В соответствии с этим в разряды PD0, PD1 и PD2 порта PORTD, настроенные на выход (линии сканирования), циклически выдаются коды сканирования, содержащие логическую 1 только в одном разряде и логические 0 в остальных.

В разряды PA0…PA3 порта PORTA, настроенные на вход без внутренних подтягивающих резисторов (линии возврата), после каждого изменения кода сканирования считывается состояние линий строк клавиатуры.

Если клавиши не нажаты, то при любом состоянии линий сканирования со всех разрядов линий возврата считываются логические 0, заданные

внешними подтягивающими резисторами R5…R7. При нажатии любой клавиши линия сканирования замыкается с линией возврата и, когда в линию сканирования с нажатой клавишей поступит логическая 1, то в соответствующей линии возврата тоже считается логическая 1.

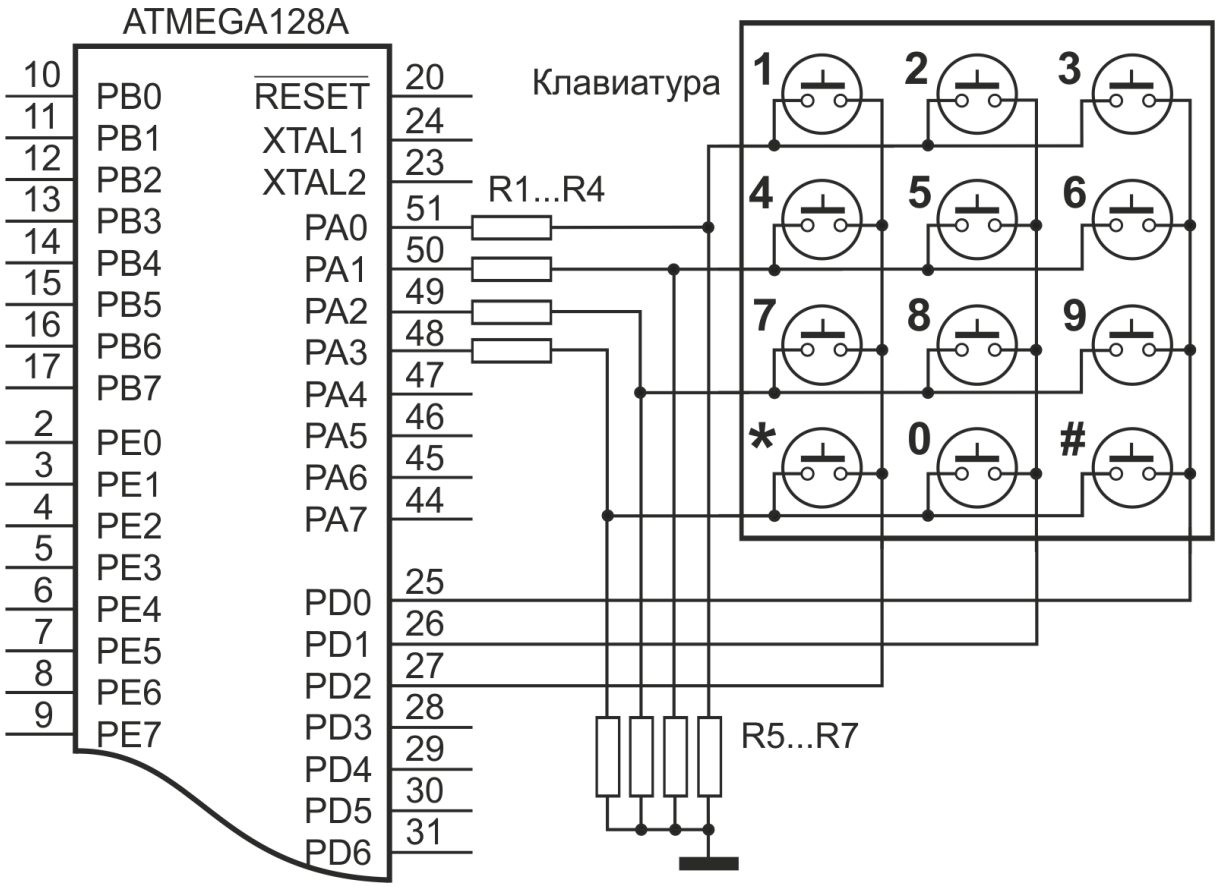


Рис. 3.26. Схема подключения клавиатуры к МК

Зная номер линии сканирования с заданной логической 1 (столбец), и номер линии возврата (строка), где появилась логическая 1, мы можем легко определить нажатую клавишу.

Есть, однако, одна особенность, которую необходимо учитывать при разработке программы. Во время работы МК с датчиками, имеющими механические контакты (кнопки, клавиши, клавиатуры и т. д.), возникает явление, называемое дребезгом. Оно заключается в том, что в момент соприкосновения или разъединения контактов сопротивление между ними хаотически меняется в течение некоторого времени, зависящего от материала, конфигурации и состояния контактов, воздействующей на контакты силы и ряда других факторов. Во время дребезга сигнал с контакта может быть прочитан как случайная последовательность нулей и единиц. Подавить это нежелательное

явление можно аппаратными средствами, но чаще это делается программным путем.

Наиболее простым способом подавления эффекта дребезга контактов является введение временно*й* задержки. Он основан на том, что программа, обнаружив первое замыкание контакта, запрещает опрос состояния этого контакта на время, заведомо большее длительности времени дребезга. Время задержки лежит в пределах 10…30 мс и подбирается экспериментально для каждого типа контактов. Повторный опрос состояния клавиши после задержки дает истинное значение этого состояния.

Окончательно алгоритм опроса состояния клавиатуры необходимо представить в виде отдельной функции. В начале этой функции разряды PD0, PD1 и PD2 порта PORTD настраиваются на выход, а разряды PA0…PA3 порта PORTA – на вход без подключения внутренних подтягивающих резисторов. Затем в ней реализуется бесконечный цикл, в котором выполняется следующая последовательность действий:

1. задать цикл формирования и записи в разряды PD0, PD1 и PD2 порта PORTD кодов сканирования с «бегающей» единицей;
2. после записи каждого кода сканирования считать и проверить состояние разрядов PA0…PA3 порта PORTA;
3. если во всех разрядах считано состояние логического 0, то закончить очередной цикл сканирования и перейти к п. 7;
4. если хотя бы в одном разряде считано состояние логической 1, то ввести паузу для подавления дребезга;
5. произвести повторное считывание разрядов PA0…PA3 порта PORTA, дающее истинное состояние нажатой клавиши;
6. по номерам столбца и строки, содержащих логическую 1, определить, какая клавиша была нажата и код нажатой клавиши вывести по команде возврата из функции;
7. конец цикла задания кода сканирования.

Варианты заданий

1. Разработать калькулятор, выполняющий операцию сложения двух целых десятичных чисел. С клавиатуры вводятся слагаемые, операция сложения реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

2. Разработать калькулятор, выполняющий операцию поразрядного логического И над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

3. Разработать калькулятор, выполняющий операцию вычитания двух целых десятичных чисел. С клавиатуры вводятся числа, операция вычитания реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

4. Разработать калькулятор, выполняющий операцию поразрядного логического ИЛИ над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

5. Разработать калькулятор, выполняющий операцию умножения двух целых десятичных чисел. С клавиатуры вводится числа, операция умножения реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

6. Разработать калькулятор, выполняющий поразрядную операцию ИСКЛЮЧАЮЩЕЕ ИЛИ над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

7. Разработать калькулятор, выполняющий операцию целочисленного деления двух целых десятичных чисел. С клавиатуры вводятся числа, операция деления реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

8. Разработать калькулятор, выполняющий операцию сдвига влево целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – число разрядов, на которое производится сдвиг, а результат выводится после нажатия клавиши #.

9. Разработать калькулятор, выполняющий операцию возведения в степень целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – показатель степени, а результат выводится после нажатия клавиши #.

10. Разработать калькулятор, выполняющий операцию сдвига вправо целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – число разрядов, на которое производится сдвиг, а результат выводится после нажатия клавиши #.

11. Разработать калькулятор, выполняющий операцию остатка от целочисленного деления двух целых десятичных чисел. С клавиатуры вводятся числа, операция вычисления остатка реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

12. Разработать калькулятор, выполняющий операцию поразрядной инверсии целых десятичных чисел. С клавиатуры вводятся число, операция инверсии реализуется нажатием клавиши #.

Проект рекомендуется реализовывать в VSCODE.

ЛР № 6 Использование ШИМ для управления аналоговыми устройствами: RGB-светодиод и звуковой излучатель

Цель работы. Программирование процедур формирования периодических звуковых и световых сигналов, а также считывание аналогового сигнала с внешних устройств (VSCODE).

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.27.



Рис. 3.27. Состояние тумблеров конфигурации стенда для работы 6

Методические указания по выполнению работы со звуковым излучателем и RGB светодиодом

В данной лабораторной установке в качестве звукового источника используется электродинамический излучатель, подключенный к разряду 4 порта PORTB, а источником света – многоцветный RGB-светодиод, соединенный с разрядами 3, 4 и 5 порта PORTE, которые по совместительству являются каналами OC3A, OC3B и OC3C таймера №3, соответственно. Схема подключения к МК RGB-светодиода и звукового излучателя показана на рис. 3.28.

Как видно из схемы подключения для вывода звуковых сигналов заданной тональности необходимо сформировать программным путем периодический сигнал в звуковом диапазоне частот с последующим выводом этого сигнала на внешний звуковой излучатель через разряд PB4 (вывод OC0 таймера №0) порта PORTB. Транзистор T4 используется в качестве усилителя мощности.

Для формирования одного периода такого сигнала необходимо в разряд порта записать логическую 1, выдержать паузу, длительность которой задает половину периода сигнала, затем записать логический 0 и опять выдержать паузу той же длительности.

Частота сформированного таким образом сигнала должна лежать в звуковом диапазоне частот. Для задания частоты целесообразно использовать системные функции задержки.

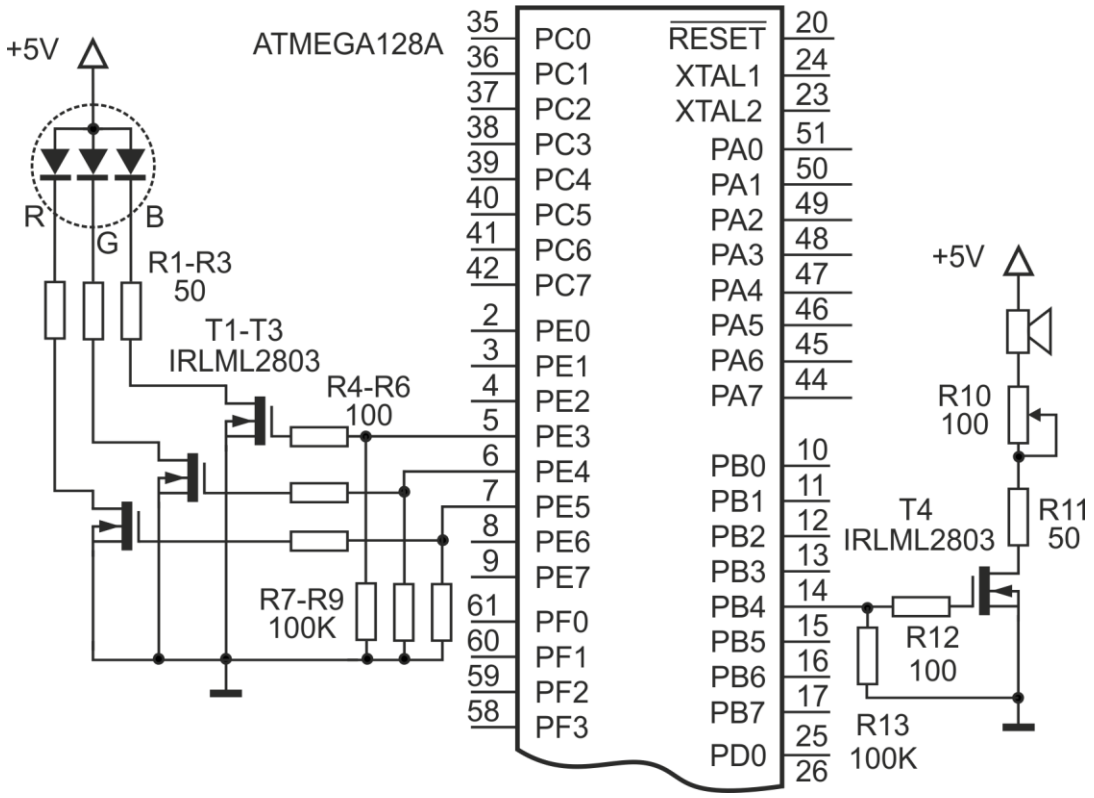


Рис. 3.28. Схема подключения RGB-светодиода и звукового излучателя

RGB (Red Green Blue) светодиод представляет собой трехкомпонентный световой излучатель, состоящий из трех расположенных на одном кристалле светодиодов красного, зеленого и синего цвета. Соотношения, связывающие цвет свечения излучателя с состоянием разрядов порта, показаны в табл. 3.11.

Таблица 3.11

Цвет свечения	**Выводы диода**	**Разряды PORTE**		
		PE3	**PE4**	**PE5**
Красный	R	0	0	1
Зеленый	G	0	1	0
Синий	B	1	0	0
Белый	R + G + B	1	1	1
Желтый	R + G	0	1	1
Фиолетовый	R + B	1	0	1
Голубой	G + B	1	1	0

Каждый из светодиодов управляется отдельно и это позволяет наряду с тремя основными цветами получать и множество других цветов путем смешения трех основных цветов различной интенсивности. Для зажигания светодиода с заданным цветом свечения необходимо подать в связанные с его выводами R, G и B разряды порта PORTE значения логической 1.

Методические указания по выполнению работы со встроенным аналого-цифровым преобразователем МК

Аналого-цифровой преобразователь (АЦП) – устройство, преобразующее входной аналоговый сигнал в дискретный код (цифровой сигнал). АЦП имеет ряд характеристик.

Разрешение АЦП — минимальное изменение величины аналогового сигнала, которое может быть преобразовано данным АЦП — связано с его разрядностью. В случае единичного измерения без учёта шумов разрешение напрямую определяется разрядностью АЦП.

Разрядность АЦП характеризует количество дискретных значений, которые преобразователь может выдать на выходе. В двоичных АЦП измеряется в битах. Например, двоичный 8-ми разрядный АЦП, способен выдать 256 дискретных значений (0…255).

Частота дискретизации характеризует частоту выборки цифровых значений из непрерывного аналогового сигнала.

АЦП в микроконтроллере ATmega128A. ATmega128A имеет встроенный 10-и разрядный АЦП последовательного приближения. К АЦП подключается 8-и канальный аналоговый мультиплексор, позволяющий производить измерения напряжения с 8-и однополярных каналов, а также с 16-и дифференциальных каналов.

Для работы с АЦП у МК AVR есть пара регистров. Регистр мультиплексора ADMUX.

Бит	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	**ADMUX**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Биты REFS*i* устанавливают опорное напряжения модуля АЦП.

REFS1	*REFS0*	*Опорное напряжение*
0	0	AREF
0	1	AVCC
1	0	Зарезервировано
1	1	Внутренний источник на 2.56 В

Результат преобразования АЦП – 10 бит, поэтому результат не может быть сохранен в одном 8-битном регистре. Результат преобразования АЦП сохраняется в двух независимых регистрах (например, результат АЦП 974 =

0b1111001110, это значение занимает 10 бит, 8 бит будут сохранены в одном регистре, а 2 бита будут сохранены в другом регистре). Регистры, в которых хранится результат преобразования АЦП – это ADCH и ADCL. ADCH сохраняет старшую часть результата, ADCL – младшую часть. Бит ADLAR регистра ADMUX определяет, как 10-битный результат преобразования АЦП будет распределяться в ADCH и ADCL. В случае, если ADLAR = 1, результат преобразования АЦП будет выровнен слева, тогда ADCH будет заполнен 8 битами результата преобразования АЦП, а ADCL будет заполнен 2 младшими битами результата преобразования АЦП и эти два бита будут занимать 6-й и 7-й бит ADCL (см. рисунок ниже). Это называется левым выравниванием.

Бит	7	6	5	4	3	2	1	0	
	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	**ADCH**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	ADC1	ADC0	-	-	-	-	-	-	**ADCL**
R/W	R/W	R/W	-	-	-	-	-	-	
Н.з.	0	0	-	-	-	-	-	-	

Если ADLAR = 0, это означает, что результат преобразования АЦП будет выровнен справа, тогда ADCL будет заполнен 8 битами результата преобразования АЦП, а ADCH будет заполнен 2 старшими битами результата преобразования АЦП, и эти два бита будут занимать 0-й и 1-й биты ADCH (см. рисунок ниже). Это называется правым выравниванием.

Бит	7	6	5	4	3	2	1	0	
	-	-	-	-	-	-	ADC9	ADC8	**ADCH**
R/W	-	-	-	-	-	-	R/W	R/W	
Н.з.	-	-	-	-	-	-	0	0	

Бит	7	6	5	4	3	2	1	0	
	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	**ADCL**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	-	-	-	-	

Пример: результат преобразования АЦП: 974 = 0b1111001110. В случае левого выравнивания ADCH = 0b11110011, ADCL = 0b10000000. В случае правого выравнивания ADCH = 0b00000011, ADCL = 0b11001110.

В случае использования левого выравнивания пользователь может прочитать только 8 бит результата преобразования АЦП в ADCH и не тратить время на чтение ADCL, в котором хранятся только два младших бита результата преобразования АЦП (в этом случае точность АЦП снижается до 8 бит).

Биты MUXi регистра ADMUX выбирают один из каналов АЦП (Atmega128A имеет 8 дифференциальных каналов АЦП).

MUX4:0	*Канал АЦП*	*MUX4:0*	*Канал АЦП*
00000	ADC0	00100	ADC4
00001	ADC1	00101	ADC5
00010	ADC2	00110	ADC6
00011	ADC3	00111	ADC7

Следующий регистр АЦП – ADCSRA.

Бит	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADFR	ADIF	ADIE	ADPS2	ADPS1	ADPS0	**ADCSRA**
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Н.з.	0	0	0	0	0	0	0	0	

Включение модуля АЦП производится установкой бита ADEN. Установка бита ADSC, запускает преобразование аналогового сигнала. Если установлен бит ADFR, модуль АЦП переходит в режим циклического перезапуска. Бит ADIF – это флаг прерывания АЦП, который устанавливается после завершения преобразования АЦП и обновления регистров данных (ADCH и ADCL). Чтобы сбросить флаг, необходимо записать в него логическую единицу. Запись логической единицы в бит ADIE разрешает прерывания от АЦП. Биты ADPSi определяют частоту дискретизации модуля АЦП.

ADPS2	*ADPS1*	*ADPS0*	*Предде-литель*	*ADPS2*	*ADPS1*	*ADPS0*	*Преддели-тель*
0	0	0	2	1	0	0	16
0	0	1	2	1	0	1	32
0	1	0	4	1	1	0	64
0	1	1	8	1	1	1	128

Ниже представлен пример функции запуска процесса преобразования АЦП и чтения 8-и битного результата.

```
uint8_t readAdcChannel(uint8_t channel) {
  // AVCC + левое выравнивание + выбор канала
  ADMUX = (1<<REFS0) | (1<<ADLAR) | channel;
  ADCSRA |= (1<<ADSC); // Начало преобразования
  // Ждем окончания преобразования АЦП
  while(!(ADCSRA & (1<<ADIF)));
  // Сброс флага завершения преобразования
  ADCSRA |= (1<<ADIF);
  return(ADCH); // возвращаем старшие 8 бит результата
}
```

Варианты заданий

При выполнении лабораторной работы необходимо написать программу считывания 10-битного результата преобразования с рукоятки ползункового потенциометра, подключённого к выводу АЦП МК ADC3. Исходя из полученного значения, нужно регулировать цвет включения RGB светодиода от начального (цвет 1) до конечного (цвет 2) значения используя ШИМ таймера №3. Помимо этого, полученное с АЦП значение должно влиять на излучаемый динамиком звуковой сигнал, либо меняя его частоту от 100 до 1000 Гц (канал OC0), либо изменяя длительность «пунктирных» звуковых сигналов от 20 до 500 мс. Распределение вариантов приведено в табл. 3.12.

Таблица 3.12

Вариант	Варьируемая величина	Цвет 1	Цвет 2
1	Частота звука	Красный	Голубой
2	Длительность «пунктира»	Зеленый	Желтый
3	Частота звука	Синий	Фиолетовый
4	Длительность «пунктира»	Красный	Белый
5	Частота звука	Зеленый	Голубой
6	Длительность «пунктира»	Синий	Желтый
7	Частота звука	Красный	Фиолетовый
8	Длительность «пунктира»	Зеленый	Белый
9	Частота звука	Синий	Голубой
10	Длительность «пунктира»	Красный	Желтый
11	Частота звука	Зеленый	Фиолетовый
12	Длительность «пунктира»	Синий	Белый

Битовые операции для целочисленных операндов

В языке СИ имеются шесть операторов (&, |, ^, ~, >>, <<), которые можно использовать в программе для манипулирования битами в целых числах.

Оператор & (поразрядное «И») выполняет операцию логического умножения отдельных одноименных разрядов двух операндов.

Оператор | (поразрядное «ИЛИ») реализует операцию логического сложения отдельных одноименных битов двух операндов.

Операторы & и |, которые работают с операндами побитно, следует отличать от логических операторов && и ||, которые оценивают каждый операнд целиком, равен он нулю или нет. Например, если переменная var1 = 1, а переменная var2 = 4, то результат побитовой операции (var1 & var2) будет равен нулю, а логической (var1 && var2) будет равен единице.

Оператор ^ (поразрядное исключающее «ИЛИ») реализует сложение по модулю 2 для одноименных разрядов.

Оператор инверсии ~ (поразрядное отрицание) инвертирует все биты переменной.

Операции сдвига перемещают разряды операнда влево (<<) или вправо (>>) на указанное число позиций.

Часто операции сдвига используют для быстрого умножения или деления операнда на степень двойки (2, 4, 8, …). Каждый сдвиг влево на один разряд приводит к умножению операнда на два, а каждый сдвиг вправо – к делению на два.

Рассмотрим более подробно битовые операции и их применение при программировании МК.

Оператор сдвига влево <<

Оператор сдвига влево **<<** выполняет поразрядный сдвиг числа влево. Его синтаксис:

```
var1 = var2 << num;
```

где var1 и var2 – целочисленные переменные, а Nun – число разрядов для сдвига.

Двоичное значение переменной var2 сдвигается влево на указанное количество разрядов num с заполнением освобождающихся справа разрядов нулями. При выходе за старший разряд сдвигаемые биты теряются.

Если var1 и var2 – одна переменная, то можно использовать укороченную форму записи

```
var <<= num;
```

Пример:

```
uint8_t n = 0x01; // n = 0b00000001
n = n << 4;       // n = 0b00010000
uint8_t m = 0x03; // m = 0b00000011
m <<= 2;          // m = 0b00001100
```

Помимо переменных можно сдвигать числовые константы, например:

```
uint8_t n;
n = 1 << 5;
```

В этом примере единица, представленная своим двоичным значением 0b00000001, сдвигается влево на пять разрядов, и переменная n принимает двоичное значение 0b00100000.

Оператор сдвига вправо >>

Оператор сдвига вправо >> выполняет поразрядный сдвиг числа вправо. Его синтаксис:

```
var1 = var2 >> num;
```

Значение переменной var2 сдвигается вправо на указанное количество разрядов num с заполнением освободившихся разрядов слева нулями. Следует отметить, что данное утверждение верно, если переменная var2 беззнакового типа, или если переменная знаковая и ее текущее значение положительное. Если переменная знаковая и ее значение отрицательное, то старшие разряды заполняются единицами. При выходе за нулевой разряд переменной сдвигаемые биты теряются.

Пример:

```
uint8_t n = 0x80; // n = 0b10000000
n = n >> 4;       // n = 0b00001000
uint8_t m = 0xC0; // m = 0b11000000
m >>= 2;          // m = 0b00110000
```

Оператор инверсии ~

Оператор ~ инвертирует каждый разряд операнда, т.е. заменяет 1 на 0 и наоборот. Этот оператор – унарный и применяется только к одному операнду.

```
var1 = ~var2;
```

Пример:

```
uint8_t n = 0xAA; // n = 0b10101010
n = ~n;           // n = 0b01010101
```

Оператор логического умножения &

Оператор & производит над операндами логическую операцию поразрядного "И". Это значит, что разряд результата будет равен 1 только в том случае, когда соответствующие разряды обоих операндов содержат 1. В противном случае он будет равен 0.

```
var1 = var2 & var3;
```

Пример:

```
uint8_t a, b, c;
a = 0x55;         // a = 0b01010101
b = 0x01;         // b = 0b00000001
c = a & b;        // c = 0b00000001
```

Оператор & часто используется для принудительной очистки заданных разрядов одного из операндов, для чего в качестве второго операнда задается так называемая маска, в которой разряды, требующие очистки, установлены в 0.

Пример:

```
uint8_t a, mask;
a = 0x75;         // a  = 0b01110101
mask = 0x0F;      // mask = 0b00001111
a = a & mask;     // a  = 0b00000101
```

Оператор логического сложения |

Оператор | производит над операндами логическую операцию поразрядного "ИЛИ". Это значит, что разряд результата будет равен 1, если 1 содержится в соответствующем разряде хотя бы одного операнда.

```
var1 = var2 | var3;
```

Пример:

```
uint8_t a, b, c;
a = 0x55;         // a = 0b01010101
b = 0x81;         // b = 0b10000001
c = a | b;        // c = 0b11010101
```

Оператор | применяется для принудительной установки заданных разрядов одного из операндов, при этом во втором операнде задается маска, в которой разряды, подлежащие установке в единицу, равны 1.

Пример:

```
uint8_t a, mask;
a = 0x05;                // a   = 0b00000101
mask = 0xF0;             // mask = 0b11110000
a = a | mask;            // a   = 0b11110101
```

Оператор сложения по модулю 2 ^

Оператор ^ производит над операндами логическую операцию исключающего "ИЛИ", при которой в каждый разряд результата записывается 1, если соответствующие разряды операндов различаются, и 0, если они совпадают.

```
var1 = var2 ^ var3;
```

Пример:

```
uint8_t a, b, c;
a = 0x55;                // a = 0b01010101
b = 0x03;                // b = 0b00000011
c = a ^ b;               // c = 0b01010110
```

Оператор ^ используется для инверсии заданных битов одного из операндов и для этого во втором операнде задается маска, в которой биты, подлежащие инверсии, установлены в 1.

Пример:

```
uint8_t a, mask;
a = 0x55;                // a   = 0b01010101
mask = 0xF0;             // mask = 0b11110000
a = a ^ mask;            // a   = 0b10100101
```

Работа с отдельными битами

В МК для задания режимов работы периферийных устройств и управления ими используются специальные регистры, каждый разряд которых привязан к конкретному устройству и задание соответствующих функций определяется содержимым этого разряда. В подобных случаях для манипуляций с битами применяют рассмотренные выше побитовые операции, при этом в качестве второго операнда чаще всего используется переменная, имеющая 1 на месте значимых бит. Остальные незначащие биты должны быть невидимыми

(замаскированными) для операций установкой 0. Поэтому указанную переменную для операций с битами часто называют маской.

Рассмотрим некоторые приемы манипуляций с битами, которые часто используются при программировании МК.

Запись единицы в заданный разряд с обнулением остальных разрядов

Самым простым и естественным способом записи одной или нескольких единиц с обнулением остальных разрядов является обычное присвоение переменной значения с единицами в требуемых разрядах и нулями в остальных, например:

```
uint8_t a, b;
a = 0x10;                // a = 0b00010000
b = 0x30;                // b = 0b00110000
```

Альтернативный и часто используемый программистами вариант основан на применении оператора сдвига влево:

```
var = 1 << num;
```

Пример:

```
uint8_t a;
a = 1 << 4;              // a = 0b00010000
```

Его легко расширить для записи в разряды переменной нескольких единиц с обнулением остальных разрядов:

```
var = (1 << num1) | (1 << num2) | ... | (1 << numN);
```

Пример:

```
uint8_t a;
a = (1 << 5)|(1 << 4); // a = 0b00110000
```

Запись единицы в заданный разряд без изменения остальных разрядов

Для записи единицы в некоторый разряд без изменения остальных разрядов можно воспользоваться маской, содержащей единицы в требуемых разрядах и выполнить над ней и исходной переменной операцию поразрядного логического "ИЛИ", например:

```
uint8_t a, mask;
mask = 0x30;             // mask = 0b00110000
a = 0x03;                // a = 0b00000011
a = a | mask;            // a = 0b00110011
```

Альтернативный вариант – применение оператора сдвига

```
var = var | (1 << num);
```

или в укороченной форме записи

```
var |= (1 << num);
```

Модификация этого способа для записи нескольких логических единиц без изменения остальных разрядов выглядит так:

```
var |= (1 << num1) | (1 << num2)|...|(1 << numN);
```

Пример:

```
uint8_t a;
a = 0x03;                    // a = 0b00000011
a |= (1 << 4) | (1 << 5);    // a = 0b00110011
```

Запись нуля в заданный разряд без изменения остальных разрядов

Для записи нуля в некоторый разряд без изменения остальных разрядов можно воспользоваться оператором:

```
var = var & ~(1 << num);
```

Эта запись равнозначна следующей:

```
var &= ~(1 << num);
```

Запись нескольких нулей без изменения остальных разрядов:

```
var &= ~((1 << num1) | (1 << num2) | ... | (1 << numN));
```

Пример:

```
uint8_t a;
a = 0x87;                    // a = 0b10000111
a &= ~((1 << 1) | (1 << 2));    // a = 0b10000001
```

Можно также применить к переменной операцию поразрядного "И" с маской, в которой в позициях обнуляемых разрядов должны быть нули.

Пример:

```
uint8_t a, mask;
mask = 0xF9;            // mask = 0b11111001
a = 0x87;               // a = 0b10000111
a = a & mask;           // a = 0b10000001
```

Инвертирование отдельных битов

Для инвертирования содержимого в заданном разряде целочисленной переменной используется операция поразрядного исключающего «ИЛИ» с помощью оператора:

```
var ^= (1 << num);
```

Инверсия нескольких разрядов без изменения остальных:

```
var ^= (1 << num1) | (1 << num2) | ... | (1 << numN);
```

Пример:

```
uint8_t a, b;
a = 0x87;                    // a = 0b10000111
a ^= (1 << 7);               // a = 0b00000111
b = 0x87;                    // b = 0b10000111
b ^= (1 << 7)|(1 << 1);      // b = 0b00000101
```

Проверка значения бита в переменной

Для проверки значения любого бита в переменной необходимо обнулить все биты, кроме проверяемого, а потом сравнить полученное значение с нулем

```
if ((var & (1 << num)) != 0 ) {
 // блок выполнится, если бит num переменной var установлен
}
if ((var & (1 << num)) == 0 ) {
 // блок выполнится, если бит num переменной var сброшен
}
```

Обмен частей переменной

Одной из типовых операций преобразования данных является обмен тетрад в байте, когда четыре младших бита помещаются на место старших, а четыре старших бита – на место младших.

Обменять тетрады любой беззнаковой байтовой переменной можно при помощи двух операций сдвига:

```
var = (var << 4) | (var >> 4);
```

В первой операции сдвига биты младшей тетрады перемещаются в старшую, а младшая тетрада при сдвиге обнуляется. Во второй операции сдвига старшая тетрада перемещается на место младшей. Затем операция поразрядного «ИЛИ» объединяет байты со сдвинутыми тетрадами.

Использование макросов

Директива #define служит для замены часто использующихся в программе констант, операторов или выражений некоторыми идентификаторами. Идентификаторы, заменяющие текстовые или числовые константы, называют именованными константами, а идентификаторы, заменяющие фрагменты программ, называют макроопределениями. Директива #define имеет две синтаксические формы:

```
#define идентификатор текст
#define идентификатор(список параметров) текст
```

Эта директива заменяет все последующие вхождения идентификатора в программе на текст.

Пример:

```
#define MIN 10
#define MAX (MIN + 20)
```

Эти директивы заменят в тексте программы каждое слово MIN на число 10, а каждое слово MAX на выражение (10+20) вместе с окружающими его скобками. Скобки, содержащиеся в макроопределении, позволяют избежать ошибок, связанных с порядком вычисления операций. Например, при отсутствии скобок выражение a = MAX*5 будет преобразовано в выражение a = 10+20*7, а не в выражение a = (10+20)*7, как это получается при наличии скобок.

При разработке компиляторов данная директива чрезвычайно широко используется для привязки адресов аппаратных ресурсов МК к их символическим именам, заданным в технической документации на МК. Если заглянуть, например, в любой заголовочный файл конкретного МК, то можно увидеть, что большая его часть состоит их директив #define. Использование заданных таким образом символических имен портов, регистров и даже отдельных разрядов, позволяет резко упростить написание программ.

При создании прикладных программ применение директивы также очень удобно и позволяет разрабатывать программы, в минимальной степени зависящие от использования конкретных аппаратных ресурсов МК.

Например, если в проекте при подключении клавиатуры использовался порт PORTA, то можно задать привязку этого подключения так

```
#define P_KEY PORTA
```

Тогда в дальнейшем в программе можно использовать символическое имя P_KEY. Если по каким-то причинам возникнет необходимость изменить

порт подключения клавиатуры, то сделать это в программе нужно будет только в одном месте – в директиве #define, а не искать места его использования по всей программе.

В директиве можно использовать и выражения, например

```
#define MASK  ((1 << 1)|(1 << 2))
```

Макросы создаются с использованием второй синтаксической формы директивы #define. При идентификаторе макроса имеется список формальных параметров, заключенных в скобки и разделенных запятыми. Формальные параметры в тексте макроопределения отмечают позиции, в которые должны быть подставлены фактические аргументы макровызова.

При вызове макроса вслед за идентификатором записывается список фактических аргументов, количество которых должно совпадать с количеством формальных параметров в макроопределении.

Пример:

```
#define MAX(x,y) (((x) > (y)) ? (x) : (y))
```

Эта директива заменит фрагмент

```
a = MAX(i, f);
```

на фрагмент

```
a = (((i) > (f) ? (i) : (f));
```

Как и в предыдущем примере, круглые скобки, в которые заключены формальные параметры макроопределения, позволяют избежать ошибок, связанных с неправильным порядком выполнения операций, если фактические аргументы являются выражениями.

При программировании МК наиболее часто применяются макросы, основанные на использовании рассмотренных выше битовых операций. Рассмотрим самые распространенные из них.

Макрос установки заданного бита в переменной или регистре:

```
#define  BIT_SET(var, bit) (var |= (1 << bit))
```

Пример:

```
BIT_SET(PORTA, 5);// установить 5-й бит PORTA
```

Макрос очистки заданного бита в переменной или регистре:

```
#define  BIT_CLR(var, bit) (var &= ~(1 << bit))
```

Пример:

```
BIT_CLR(PORTC, 4);// очистить 4-й бит PORTC
```

Макрос инверсии заданного бита в переменной или регистре:

```
#define  BIT_INV(var, bit) (var ^= (1 << (bit))
```

Пример:

```
BIT_INV(PORTB, 1);// инвертировать 1-й бит PORTB
```

Макрос возвращает 1, если заданный бит в переменной или регистре равен 1:

```
#define  IS_BIT_SET(var, bit) ((var & (1 << bit)) != 0)
```

Пример:

```
if(IS_BIT_SET(PORTB, 1)) {Блок оператора if};
```

Макрос возвращает 1, если заданный бит в переменной или регистре равен 0:

```
#define  IS_BIT_CLR(var, bit) ((var & (1 << bit)) == 0)
```

Пример:

```
if(IS_BIT_CLR(PORTB, 1)) {Блок оператора if};
```

Данные макросы можно поместить в заголовочный файл (например, macros.h) и использовать его в своих программах. При появлении новых макросов можно легко расширять библиотеку, помещая их в этот файл.

Правила построения блок-схем

Для визуального отображения работы программы используются блок-схемы. Блок-схемы отображают пошаговое выполнение программы или любого другого процесса, другими словами, они представляют алгоритм работы программы схематично. При составлении блок-схем ключевым моментом является создание схемы таким образом, чтобы она позволяла максимально быстро разобраться в работе программы, исправить неточности при необходимости, а также была понятна любому другому специалисту в данной области. В связи с данными аспектами блок-схемы разрабатываются по установленным стандартам и правилам. В данных стандартах описываются конкретные блоки для переменных, операций. Основные правила для составления блок-схем прописаны в ГОСТ 19.701-90.

В соответствии с ГОСТ выделяются следующие основные блоки для их использования в блок-схемах.

Блок «Пуск-останов»

Вид данного блока представлен на рисунке а. Рассматриваемый блок используется для обозначения блоков начала и конца программы, а также при описании функций. Причем, в случае добавления данных блоков в главную программу в начале и конце описание внутри блоков «Начало» и «Конец» соответственно. В случае использования данных блоков для описания функции в начале и конце используются надписи «Вход» и «Выход» или «Возврат переменной» соответственно.



Рис. 3.26. Блок «Пуск-останов»

Блок данных

Для ввода или вывода различных типов данных используется блок, представленный на рисунке 5.2.



Рисунок 5.2. Блок «Ввода-вывода»

Блок процесса

Для описания той или иной операции, выполняемой в программе, используется блок, представленный на рисунке 5.3. При описании одной операции используется один блок. В случае использования нескольких операций последовательно добавляется несколько блоков соответственно.



Рисунок 5.3. Блок процесса

•Блок подпрограммы

При наличии в программе функций для описания алгоритма их работы также должны создаваться блок-схемы. Вызов функции в основной программе осуществляется с помощью блока, представленного на рисунке 5.4. Внутри данного блока записывается название функции, принимаемые и возвращаемые переменные, используемые непосредственно в главной программе.



Рисунок 5.4. Блок предопределённого процесса

•Блок проверки условия

При использовании в программе проверки условий с двойным или множественным выбором используют блок, изображенный на рисунке 5.5. На рисунке 5.6 можно увидеть пример использования данного блока для случая с двумя выходами (слева) и для случая с множественным выбором (справа).

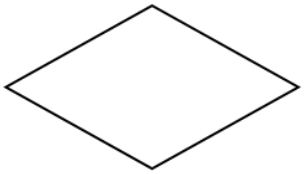


Рисунок 5.5. Блок решения

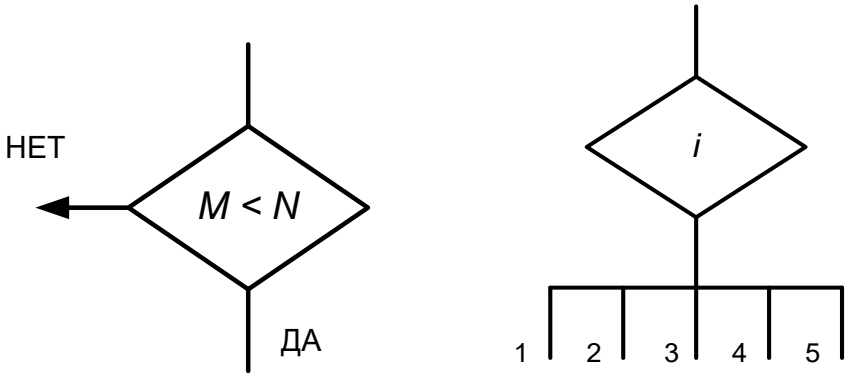


Рисунок 5.6. Примеры использования блока решения

•Блок для цикла со счетчиком

Для обозначения цикла со счетчиком используется отдельный блок, показанный на рисунке 5.7. Однако данный цикл также можно реализовать с помощью блока условия и блоков процесса.



Рисунок 5.7. Блок для цикла со счетчиком

•Общие блоки для циклов

В общем случае для циклов с предусловием и постусловием можно использовать блоки, представленные на рисунке 5.8. На рисунке 5.9 показан пример использования рассматриваемых блоков. Но также циклы можно описывать блоками условия и блоками процесса.

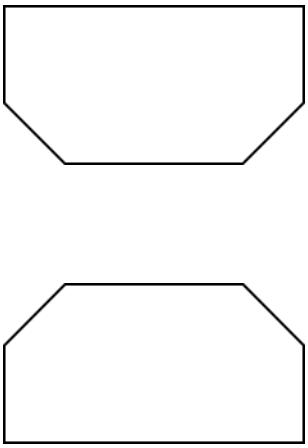


Рисунок 5.8. Блоки для циклов

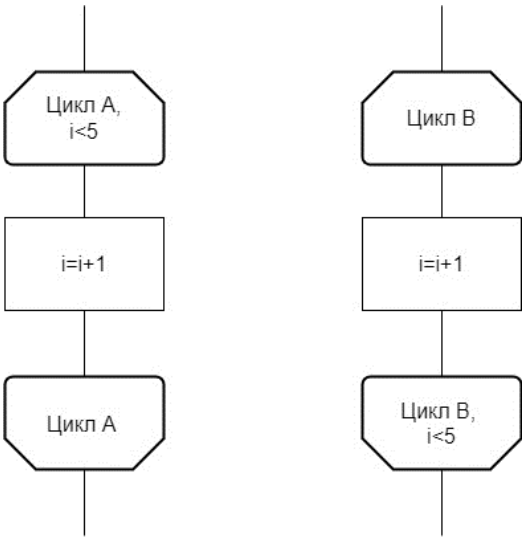


Рисунок 5.9. Пример использования блоков для циклов

•Блок соединителя

В случае, если вся блок-схема не помещается на одном листе или же если соединение некоторых линий мешает читабельности блок-схемы, можно разрывать части схемы с использованием блока соединителя в виде, представленном на рисунке 5.10. Пример использования данного блока можно увидеть на рисунке 5.11.

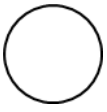


Рисунок 5.10. Блок соединителя

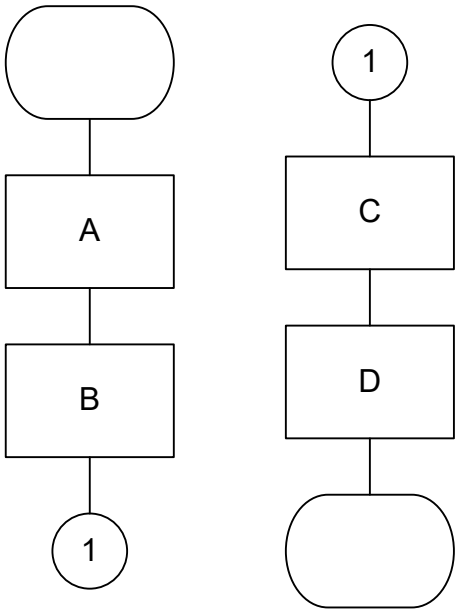


Рисунок 5.11. Пример использования блока соединителя

•Блок комментария

При добавлении комментария к некоторой части схемы следует воспользоваться блоком, который показан на рисунке 5.12.

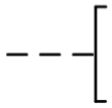


Рисунок 5.12. Блок комментария

Примечания:

1. Высота блоков должна выбираться из ряда 10, 15, 20 мм. Допускается увеличивать высоту блоков на число, кратное 5 мм. Ширина блоков должна быть в 1.5 раза больше высоты (ширина = 1.5 * высота).

2. Все блоки соединяются сплошными линиями. В случае стандартного направления линий, а именно – слева направо и сверху вниз, дополнительных стрелок не требуется. В случае нестандартного направления линий, а именно – справа налево и снизу вверх, в конце линий ставится стрелка, указывающая направление движения.

3. В схемах следует избегать пересечений линий. В местах, где этого избежать невозможно, следует использовать разрыв с блоками соединителями.

4. Все блоки следует изображать примерно одного размера. Тексты внутри блоков должны иметь одинаковый шрифт и размер. Вписываемый

текст необходимо добавлять максимально кратко, но без потери смысла текста. В случае, когда требуемый текст не вмещается в блок, можно добавить комментарий к блоку.